



Mohamed Shathir Sheriff

2526110

Havelock Smart Resident Portal: An AI-Driven Apartment Management System

BSc (Hons) Software Engineering
Undergraduate Thesis Report
Department of Computer Science & Technology

Mrs. Rameesha Kankanamge

AY 2025/2026

Abstract

The Havelock Smart Resident Portal is a specialized full-stack ecosystem engineered to modernize communal living within the Havelock City apartment complex. While conventional estate management solutions often provide generic functionality, this system addresses specific localized friction points – namely, communication latencies, unintelligible maintenance tracking, and inefficient visitor and parcel logistics.

At its core, the platform introduces a predictive AI-driven complaint engine that autonomously categorizes and prioritizes maintenance requests based on urgency, ensuring critical infrastructure issues receive immediate attention. The architectural framework utilizes a React.js and Tailwind CSS frontend for a high-performance user experience, supported by a scalable Node.js, Express.js and MongoDB backend. Unique features, such as automated 24-hour digital visitor credentials and a unified inter-vehicle communication bridge, create a seamless operational flow between residents, reception and security personnel. By digitizing the administrative lifecycle, the application eliminates manual paperwork, enhances operational transparency, and establishes a new benchmark for AI-integrated smart living within high-density residential communities.

Acknowledgement

I would like to deeply appreciate Mrs. Rameesha Kankanamge, my supervisor, for her important advice, patience, and constant assistance during the development of this project. Her knowledge and insightful feedback were crucial in forming the Havelock Smart Resident Portal.

I am also grateful to our course coordinator Dr. Gayana Fernando and Senior students at the University of Bedfordshire for providing necessary resources and environment to conduct this research.

Finally, I would like to thank my family and friends for their constant support and encouragement, which kept me motivated during this journey.

Dedication

This project is dedicated to my family, whose passionate affection, sacrifices have motivated me to keep working toward my goals. I also extend my heartfelt gratitude to my supervisor, lecturers and friends for their invaluable guidance and support, which encouraged me to complete this journey and strive for excellence.

List of Keywords

Table 1. List of Keywords

Keyword	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheet
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	Jason Web Token
JSON	JavaScript Object Notation
ML	Machine Learning
ODM	Original Design Manufacture
OAuth	Open Authorization
UI	User Interface
UX	User Experience
UML	Unified Modelling Language
URL	Uniform Resource Locator
WBS	Work Breakdown Structure

Table of Contents

Abstract.....	i
Acknowledgement.....	ii
Dedication.....	iii
List of Keywords.....	iv
List of Figures.....	vii
List of Tables.....	ix
Chapter 01: Introduction.....	1
1.1 Project Background.....	1
1.2 Project Aims & Objectives.....	1
1.3 Research Questions.....	2
1.4 Description of the Artefact.....	3
1.5 Structure of the Report.....	3
Chapter 02: Literature Review.....	5
2.1 Introduction.....	5
2.2 Existing Systems & Their Limitations.....	5
2.3 AI-Based Complaint Classification & Prioritization.....	6
2.4 Digital Visitor Management.....	6
2.5 User Experience & Role-Specific Access in Smart Portals.....	6
2.6 Research Gap & Conclusion.....	6
Chapter 03: Methodology.....	9
3.1 Research Approach.....	9
3.2 Agile Methodology.....	9
3.2.1 Sprint Planning & Execution.....	9
3.3 Requirement Gathering Methods.....	10
3.3.1 Primary Data: Resident Survey.....	10
3.3.2 Secondary Data: Current Systems & Literature Review.....	10
3.4 Tools & Technologies Used.....	10
3.5 Time Planning.....	11
3.6 Risk & Mitigation.....	12
Chapter 04: System Analysis & Design.....	14
4.1 Functional & Non-functional Requirements.....	14
4.2 Use Case Diagram.....	15
4.3 System Architecture.....	16
4.4 ER Diagram.....	17
4.5 Class Diagram.....	18
4.6 UI/UX Design.....	19
4.7 Network Diagram.....	23
Chapter 05: System Implementation.....	25
5.1 Frontend Development.....	25
5.1.1 Technology Stack.....	25
5.1.2 Monorepo Layout & Responsibilities.....	26
5.1.3 Routing & Global Providers.....	26
5.2 Backend Development.....	26
5.2.1 Technology Stack.....	26
5.2.2 High Level Architecture.....	27
5.2.3 Data Models & Identifiers.....	28
5.2.4 Domain Logic Highlights.....	28
5.3 Database Integration.....	28

5.4 AI Module Implementation	30
5.4.1 System Overview: Simple AI Rule-Based Logic Method.....	30
5.4.2 The Logic.....	30
5.4.3 Implementation & Data Flow	31
Chapter 06: Testing & Evaluation	33
6.1 Introduction to Testing	33
6.2 Testing Methodology	33
6.3 Test Cases	34
6.3.1 User Authentication.....	34
6.3.2 Complaint Management	38
6.3.3 Visitor Management	43
6.3.4 Inter-Vehicle Booking Management.....	49
6.3.5 Parcels Management.....	51
6.3.6 Billing Management	55
6.3.7 Accounts Management	58
Chapter 07: Results & Discussion	65
7.1 System Performance & Workflow Efficiency	65
7.2 Intelligent Features & Decision Support	66
7.3 User Experience and System Evaluation.....	67
7.4 Limitations & Future Improvements	68
7.5 Final Insights.....	68
Chapter 08: Conclusion	69
References.....	70
Appendixes.....	71
Appendix A – Similar Applications	71
Appendix B – Recording of the System Demonstration.....	71
Appendix C – Project Poster	72

List of Figures

Figure 1. Agile Methodology.....	9
Figure 2. WBS	11
Figure 3. Gantt Chart	12
Figure 4. Use Case Diagram	15
Figure 5. System Architecture	16
Figure 6. ER Diagram	17
Figure 7. Class Diagram	18
Figure 8. Home Page.....	19
Figure 9. Authentication Interface	20
Figure 10. Resident Dashboard.....	20
Figure 11. Admin Dashboard.....	21
Figure 12. Reception Dashboard.....	21
Figure 13. Security Dashboard	22
Figure 14. User Profile.....	22
Figure 15. Notification Interface.....	23
Figure 16. Network Diagram	23
Figure 17. Backend Folder Structure	27
Figure 18. Connection Setup.....	28
Figure 19. Models Barrel	29
Figure 20. Admin's Security Page Complaint Priority Rules	31
Figure 21. The Live Feedback	32
Figure 22. Evidence 01	34
Figure 23. Evidence 02	34
Figure 24. Evidence 03	35
Figure 25. Evidence 04	35
Figure 26. Evidence 05	36
Figure 27. Evidence 06	36
Figure 28. Evidence 07	37
Figure 29. Evidence 08	37
Figure 30. Evidence 09	38
Figure 31. Evidence 10	38
Figure 32. Evidence 11	39
Figure 33. Evidence 12	39
Figure 34. Evidence 13	40
Figure 35. Evidence 14	40
Figure 36. Evidence 15	41
Figure 37. Evidence 16	41
Figure 38. Evidence 17	42
Figure 39. Evidence 18	42
Figure 40. Evidence 19	43
Figure 41. Evidence 20	43
Figure 42. Evidence 21	44
Figure 43. Evidence 22	44
Figure 44. Evidence 23	45
Figure 45. Evidence 24	45
Figure 46. Evidence 25	45
Figure 47. Evidence 26	46
Figure 48. Evidence 27	46

Figure 49. Evidence 28	47
Figure 50. Evidence 29	47
Figure 51. Evidence 30	48
Figure 52. Evidence 31	48
Figure 53. Evidence 32	49
Figure 54. Evidence 33	49
Figure 55. Evidence 34	50
Figure 56. Evidence 35	50
Figure 57. Evidence 36	51
Figure 58. Evidence 37	51
Figure 59. Evidence 38	52
Figure 60. Evidence 39	52
Figure 61. Evidence 40	53
Figure 62. Evidence 41	53
Figure 63. Evidence 42	54
Figure 64. Evidence 43	54
Figure 65. Evidence 44	55
Figure 66. Evidence 45	55
Figure 67. Evidence 46	56
Figure 68. Evidence 47	56
Figure 69. Evidence 48	57
Figure 70. Evidence 49	57
Figure 71. Evidence 50	58
Figure 72. Evidence 51	58
Figure 73. Evidence 52	59
Figure 74. Evidence 53	59
Figure 75. Evidence 54	60
Figure 76. Evidence 55	60
Figure 77. Evidence 56	61
Figure 78. Evidence 57	61
Figure 79. Evidence 58	62
Figure 80. Evidence 59	62
Figure 81. Evidence 60	63
Figure 82. Evidence 61	63
Figure 83. Evidence 62	64
Figure 84. Evidence 63	64
Figure 85. Complaint Workflow	66

List of Tables

Table 1. List of Keywords	iv
Table 2. Features between Current Systems and Proposed Project	7
Table 3. Risk & Mitigation	13
Table 4. Functional Requirements	14
Table 5. Non-functional Requirements	14
Table 6. Frontend Tech Stack	25
Table 7. Monorepo Responsibilities	26
Table 8. Backend Technology Stack	26
Table 9. Test Case 01	34
Table 10. Test Case 02	37
Table 11. Test Case 03	38
Table 12. Test Case 04	40
Table 13. Test Case 05	42
Table 14. Test Case 06	43
Table 15. Test Case 07	44
Table 16. Test Case 08	46
Table 17. Test Case 09	48
Table 18. Test Case 10	49
Table 19. Test Case 11	51
Table 20. Text Case 12	52
Table 21. Test Case 13	55
Table 22. Test Case 14	57
Table 23. Test Case 15	58
Table 24. Test Case 16	60
Table 25. Test Case 17	61
Table 26. Test Case 18	63
Table 27. Summary of Key Features & Outcomes	65
Table 28. Performance Comparison	65
Table 29. AI Prioritization Examples	66
Table 30. Role-Based User Feedback	67

Chapter 01: Introduction

This chapter presents the Havelock Smart Resident Portal project, an AI-driven web-based platform developed to modernize and streamline residential management for Havelock City apartments. It starts by describing the project background and the challenges residents face with traditional, manual communication and management techniques. The project aims and objectives along with the primary issues seeks to address, such as delays in visitor handling and maintenance responses. The chapter also explains the artefact itself, discussing key features including AI-powered complaint prioritization, a 24-hour digital visitor pass system, billing system, and parcel tracking. It clearly outlines the system's capabilities, such as offering dedicated dashboards for residents, admin, reception and security. In the end, it gives an overview of the report structure, so readers know what to expect in the next chapters.

1.1 Project Background

In many residential apartment communities in Sri Lanka, communication between residents & management remains inconsistent and inefficient. Daily tasks depend on smooth coordination among residents, security personnel, reception staff, and administrators; however many towers still use manual records or fragmented digital tools to manage visitor access, complaints, parcel handling, and billing updates. This creates delays, limited transparency, and frequent information gaps across stakeholders, highlighting the need for a unified and reliable-communication-driven management approach.

Currently, residents at Havelock City face several challenges:

- **Manual Communication:** Conducting phone calls or physical visit to the office are two time-consuming and unsuccessful ways to communicate with management.
- **Visitor Management:** Security gates are often relied on paperwork and manual entry logs, which can result in serious delays and problems with security.
- **Maintenance Delays:** Maintenance complaints are usually resolved in the order they are received without a system to identify and prioritized major problems.

While, general apartment applications like ApnaComplex or MyGate exist, they often lack intelligent automation features and real-time coordination across all residential departments. The system proposed as specialized, AI-driven solution to address these gaps.

1.2 Project Aims & Objectives

This project proposes the development of an AI-enabled apartment management platform integrated with security-gate operations to improve communication efficiency, process automation, and resident safety in the Havelock City apartment environment.

The objectives are,

- Design and develop a smart web-based portal integrating resident, reception, security and admin functionalities.

- Provide a role-based workflows so each stakeholder sees only the tools relevant to their responsibilities.
- Improve operational efficiency and response time by replacing manual coordination with connectional portal flows.
- Ensure end-to-end traceability from resident request to staff action and status resolution.
- Deliver real-time visibility and communication through centralized notification and dashboard summaries.
- Support data-driven administration with exportable reports (PDF) for monitoring, audits and decision making.
- Maintain a foundation that is scalable, secure and maintainable for larger communities and future enhancements.
- Develop an AI-based complaint prioritization system using rule-based technique.
- Create digital visitor management with 24-hour pass.
- Apply online billing and tracking for transparency.

1.3 Research Questions

These are some specific questions researcher mainly focus to build the Havelock Smart Resident Portal and how solved it.

- 1 How can apartment operations be digitalized end-to-end?**
Built a role-based monorepo with dedicated portals connected to shared backend APIs.
- 2 How can resident identity and role access be handled securely?**
Built unified login/signup flow with role direct, resident guard logic, and operational Google sign-in.
- 3 How can visitor handling be made fast and traceable?**
Built 6-digit visitor pass generation, gate verification, entry/exist recording, and pass lifecycle actions.
- 4 How can parcel intake and resident updates be improved?**
Built parcel registration, status tracking, resident-side pickup flow, and parcel notification.
- 5 How can complaints prioritize and resolved efficiently?**
Built complaint submission, AI-assisted priority logic, status lifecycle updates, and admin triage workflows.
- 6 How can billing visibility and payment tracking be improved?**
Built bill generation/listing, resident-scoped bill views, payment status updates, and billing related notifications.

- 7 How can communication across roles remain timely?**
Built API-back notifications with role-specific filtering, unread indicators, mark-read/clear actions, and bell integration.
- 8 How can operational outcomes be reviewed and reported?**
Built downloadable PDF reports for complaints, billing, and visitor logs with date filtering.
- 9 How can account governance be managed centrally?**
Built account management for residents/admin/reception/security with profile and security settings per role.
- 10 How can system usability support real operations?**
Built consistent navigation, standardized page headers/icons, dashboard summaries, and workflow-oriented screens.

1.4 Description of the Artefact

The project artefact is a web-based apartment operations platform developed for Havelock City to support integrated resident and management services. The system combines complaint handling, billing visibility, parcel workflows, visitor access, and notifications within a role-oriented architecture. A key component is an AI-assisted complaint-priority mechanism that classifies requests by urgency to support faster operational response.

Main functional components:

- Dedicated dashboards for Resident, Admin, Reception and Security users.
- An AI-supported complaint prioritization. (Urgent, Medium, Low)
- Time-limited digital visitor pass generation (24-hour validity).
- Billing management and payment status tracking.
- Inter-vehicle booking workflow.
- Parcel registration, tracking and notification updates.
- Secure authentication with role-based access control.

1.5 Structure of the Report

The thesis report has 08 chapters.

- 1 Introduction:** This chapter outlines the project context, defines the intended artefact, and presents the key aims and objectives that guide the proposed work.
- 2 Literature Review:** The second chapter discuss about the existing systems and their limitations of the research gap.
- 3 Methodology:** The systematic approach to the study and the development of the proposed project is explained in this chapter. It describes the technological structures

and tools used to create the final AI-driven solution, the methods used to gather resident needs, and the framework selected to manage the project.

- 4 **System Analysis & Design:** This chapter give a clear idea about the functional & non-functional requirements, UML diagrams & UI/UX design.
- 5 **System Implementation:** Chapter five explain how the frontend, backend, database and AI module implemented by the researcher.
- 6 **Testing & Evaluation:** This chapter discuss the test cases for each feature, and analyses what worked well and what didn't.
- 7 **Results & Discussion:** This chapter explains what the system achieved, how well the features work, Comparison with existing systems and benefits of the system.
- 8 **Conclusion:** This chapter give a summary of project achievements and future improvements.

Chapter 02: Literature Review

2.1 Introduction

The evaluation of residential living has been significantly accelerated by digital integration, particularly in the realms of community oversight and service efficiently. Since many systems for estate management concentrate on fundamental features like announcement or payment tracking, there remain an absence of technologies that offer seamless, AI-driven resident engagement. This chapter examines recent studies and innovations that are relevant to Havelock Smart Resident Portal with an emphasis on secure role-based access, digital visitor management, parcel tracking and AI-based complaint handling.

With a focus on the growing need for regular updates, expert control, and seamless communication between residents, security staff, and reception, the review examines improvements in technology and user experience in residential management systems. It also highlights the shortcomings of the existing systems, which do not include automatic visitor pass production or sophisticated complaint classification. By researching existing solutions and identifying these limitations, this offers a strong base for creating an AI-driven, completely combined resident management system that is suitable for the benefits of the apartments.

2.2 Existing Systems & Their Limitations

Nowadays systems, such as ApnaComplex and MyGate, provide technical innovations for common household tasks like visitor logging and billing, but they have a number of significant limitations.

The primary limitations found in current systems are combined as follows:

- **Lack of Intelligent Prioritization:** Current system like ApnaComplex allow residents to file complaints, but they do not provide AI-based prioritization. All problems must be solved manually by management, which often leads to major delays in fixing critical problems such as water leaks or electrical failures.
- **Manual Data Entry and Collaboration:** Many systems allow residents to manually type information, and real-time changes are often not linked across many departments like reception, security, and management.
- **User Experience Fragmentation:** Current systems often offer unconnected features across random interfaces. For example, residents could need to use separate modules or communication channels for billing, parcel updates, and visitor approvals, rather than a single unified site.
- **Lack of Decision Assistance:** Most widely used existing platforms still provide limited intelligent decision support and do not consistently include capabilities such as automated complaint prioritization or predictive issue handling, both of which are important for efficient management of large-scale apartment communities.

The Havelock Smart Resident Portal aims to offer a more effective and user-friendly experience than current options by addressing these gaps using AI-driven automation and a unified service management interface.

2.3 AI-Based Complaint Classification & Prioritization

AI supports key operational tasks such as classification, prediction, and decision support, which is why it has become widely adopted in digital service systems. In complaint management, AI-driven prioritization helps organization process high volumes of requests by identifying urgency levels and assigning issue categorize in a more consistent way. Prior studies show that rule-based models can effectively group complaints into severity bands such as urgent, medium and low. When such prioritization is integrated into service portals, response handling becomes faster, critical issues are escalated earlier, and inconsistency caused by manual judgement is reduced. However, many existing resident platforms still rely on limited or static classification methods, which can delay service resolution and reduce operational efficiency.

2.4 Digital Visitor Management

Visitor handling is a critical part of apartment security & front-desk operations. In many residential settings, visitor records are still maintained through manual registers, which can be slow, inconsistent, and prone to entry errors. Recent studies emphasize that digital visitor-systems improve control through features such as pre-authorized access, faster check-in, and automatically generated visitor credentials. Although some current platforms support QR-based verification, capabilities like time-limited pass issuance, coordinated updates between reception and gate security, and real-time status tracking are still not consistently implemented. The weak integration between reception desks and security checkpoints in many existing solutions causes delays and reduces operational clarity. These limitations indicate the need for a more integrated, automated, and responsive visitor management approach.

2.5 User Experience & Role-Specific Access in Smart Portals

The effectiveness of a web-based apartment platform depends heavily on two elements that are user experience and role-specific access control. Existing studies show that clear navigation, intuitive visual design, and easy-to-read dashboards improve usability and reduce the time required for user onboarding. In a smart apartment context, separate interface for Admin, Resident, Reception and Security users allow each group to view only the functions relevant to their responsibilities, which improves task efficiency and minimize operational errors. Research also highlights that secure authentication mechanisms, such as JWT-based session managing, strengthen across protection and help safeguard resident-related data. Despite these improvements in recent systems, many currently used platforms still provide generic interfaces with limited personalization, resulting in lower usability and slower completion of daily operational tasks.

2.6 Research Gap & Conclusion

This study addresses several unresolved gaps in research on digital apartment management and smart living systems. While many existing platforms provide basic platforms provide basic functions such as visitor logs, maintenance billing, and announcements, they often lack deeper automation and coordinated communication among residents, reception teams, and security staff. A key shortcoming is the limited use of AI-supported complaint prioritization,

which is important of ensuring that high-risk issues are identified early and handled without delay.

In addition, prior work indicates that fully integrated residential service portals are still uncommon. In many current solutions, visitor management, parcel handling, billing, and complaint workflows remain separated across different tools or channels. As a result, residents frequently depend on phone calls, paper-based notes, or informal messaging applications for follow-up.

Table 2. Features between Current Systems and Proposed Project

Feature	ApnaComplex	MyGate	Havelock Smart Resident Portal (Proposed Project)
AI Complaint Prioritization	No	No	Yes
24-Hour Visitor Pass	No	No	Yes
Parcel Tracking System	Yes	Yes	Yes
Online Billing	Yes	Yes	Yes
Inter-vehicle booking	No	No	Yes
Unified Single Portal	No	No	Yes
Role-based Dashboards	Yes	Yes	Yes

Research Gaps summary:

- **AI-Driven Complaint Prioritization:** Many current apartment systems still process complaints in submission order without intelligence urgency assessment. As a result, high-impact issues such as water leaks, electrical faults, or security-related incidents may experience avoidable response delay. The lack of structured, configurable priority logic limits operational efficiency and reduces service responsiveness. This reveals a clear need for an intelligent complaint workflow that classifies requests by severity and supports faster administrative action.
- **Time-limited Digital Visitor Access and Gate verification:** Although some platforms support visitor approval, they often don't provide integrated real-time gate validation with automatically generated short-validity passes. In practice, many communities continue relying on phone confirmation or manual register entries at the gate, which increases wait time and weakens process consistency. A digital, time-bound pass mechanism with direct security verification is therefore a significant missing capability.
- **Centralized Multi-Service Residential Management:** Existing solutions are frequently fragmented, with billing, visitor handling, parcel updates, and complaints distributed across separate modules or communication channels. This separation

creates user friction, duplicated effort, and dependency on manual follow-up. The absence of a unified platform remains a major limitation in current apartment management ecosystems.

This project is designed to bridge the gap between conventional apartment operations and the growing demand for integrated smart-living systems. By combining AI-based complaint prioritization, 24-hour digital visitor pass workflows, and centralized service management in one platform, Havelock Smart provides a more modern, efficient, and practical approach for apartment communities.

Chapter 03: Methodology

3.1 Research Approach

This project uses a mix-method approach, which combines qualitative and quantitative research.

- **Quantitative Research**

A standard survey was used to gather data from apartment residents to identify common typical problems, such as maintenance issues and guest delays.

- **Qualitative Research**

To identify functional gaps, a comparative analysis of current systems like MyGate and ApnaComplex was conducted, with a special focus on the lack of automated prioritization and unified communication channels.

3.2 Agile Methodology

The Havelock Smart Resident Portal was developed following the Agile Methodology, which emphasizes flexibility and incremental progress. This method was worked really well for integrating the components with React-based frontend. Agile was selected over the traditional Waterfall model to allow for continuous refinement.

3.2.1 Sprint Planning & Execution

The project was divided into separate sprints, each lasting roughly two weeks. Following the WBS (Work Breakdown Structure), key milestones were achieved in stages:

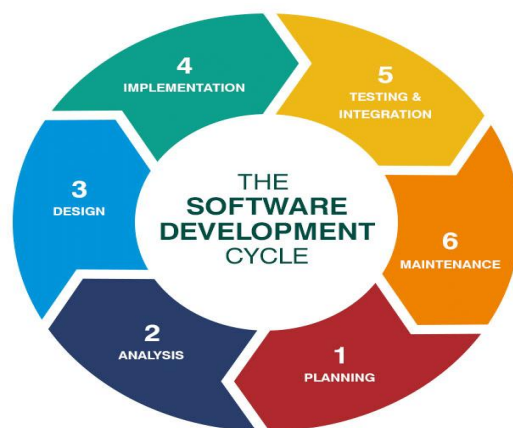


Figure 1. Agile Methodology

- **Sprint 01:** Setting up the MERN environment, configuring the Node.js server, and designing the MongoDB schemas.
- **Sprint 02:** Added advanced features including API endpoints for the 24-hour digital visitor pass system, data visualization for monthly billing and automated parcel delivery notifications.

- **Sprint 03:** Integrating simple AI rule-based logic into the backend to handle maintenance complaint severity.
- **Sprint 04:** Focused on testing and fixing bugs.

3.3 Requirement Gathering Methods

3.3.1 Primary Data: Resident Survey

The researcher collected primary data through an online survey distributed aimed at residents to uncover real-world challenges in apartment living. Participants provided feedback on how they currently navigate visitor entry, track deliveries, report issues and handle monthly payments. These insights served as the foundation for the research, make sure the final product effectively addresses the most significant user concerns.

3.3.2 Secondary Data: Current Systems & Literature Review

Secondary data is collected by reviewing current apartment management systems, such as ApnaComplex and MyGate, and reviewing research papers. This helps in determining the capabilities, limitations, and research gaps of the current apartment management system.

3.4 Tools & Technologies Used

- **Frontend: React.js & Tailwind CSS**
The UI was developed using React.js to implement a fast, responsive, and component-based experience for residents and staff. Tailwind CSS was utilized for styling to make sure a modern and user-friendly design across all dashboards.
- **Backend: Node.js & Express.js**
The server-side logic was built using Node.js and the Express.js technology. These technologies manage the essential features of the application, including the AI logic, visitor pass API endpoints, and billing.
- **Database: MongoDB & MongoDB Atlas**
To store flexible data structure including resident profiles, visitor parcels, and parcel details, MongoDB was selected as the NoSQL database. For reliable and safe cloud-based data storage, the database is hosted on MongoDB Atlas.
- **AI Integration**
The system integrates AI technologies for the purpose of intelligently classifying and prioritization of maintenance complaints. This logic allows the system to automatically classify problems as High, Medium or Low severity.
- **Development Tools**
Cursor was used as the primary IDE for coding and debugging. GitHub was used for version control to track development progress and maintain code integrity.

3.5 Time Planning

To keep the project on track, scheduling was essential. The project was broken down into smaller, manageable tasks using a WBS, assigning priorities, monitoring progress, and determining dependencies became easier as a result.

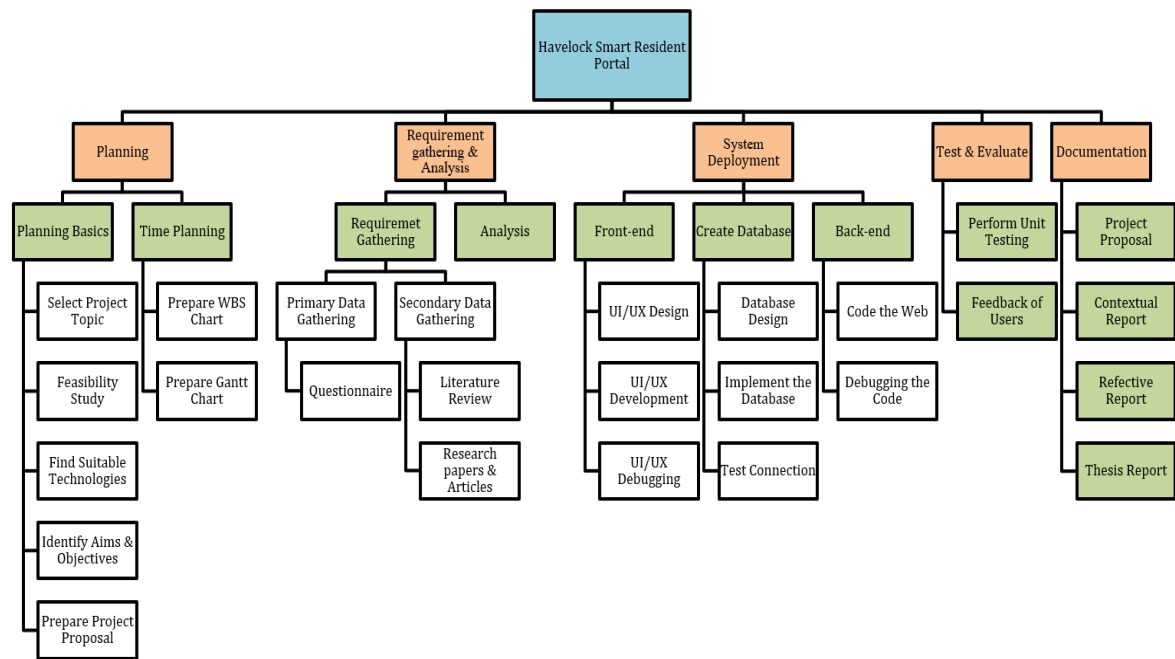


Figure 2. WBS

The project timeline, deadlines, and connection between different tasks were presented using a Gantt chart. The main goals were:

1. Project Initiation and Planning.
2. Requirement Analysis & Research.
3. System Design.
4. Core Development.
5. Testing & Evaluation.
6. Final Deployment & Documentation.

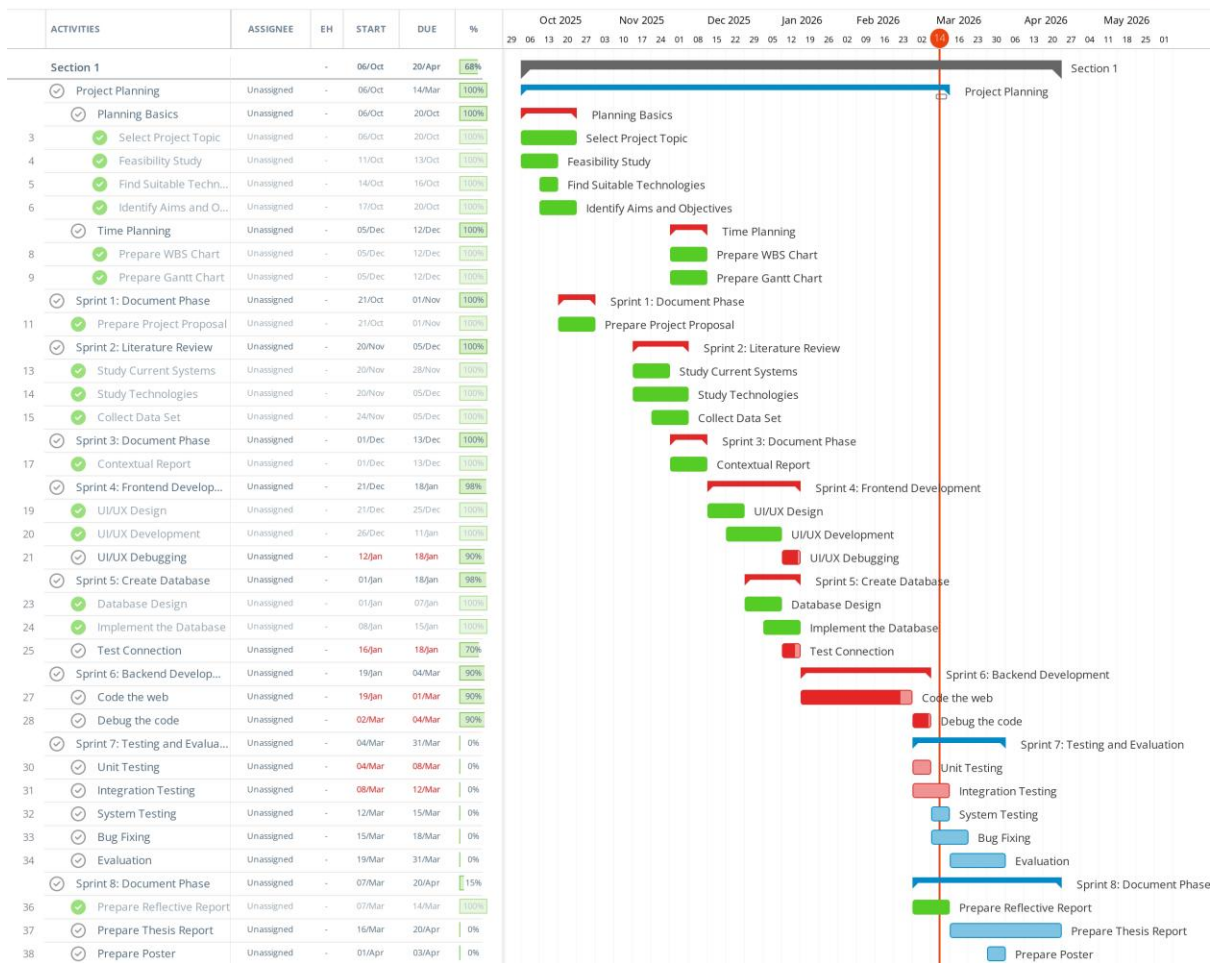


Figure 3. Gantt Chart

3.6 Risk & Mitigation

There may be challenges related with the Havelock Smart Resident Portal development that could affect the project performance. Early identification of these risks is crucial, followed by the beginning of realistic mitigation strategies to minimize impacts.

The below table displays about the risk analysis and potential mitigation actions that can be taken.

Table 3. Risk & Mitigation

Risk	Mitigation Action
Secretes in environment files or repo.	Avoid committing environment files.
MongoDB unavailable.	Health check endpoint.
SMTP misconfiguration.	Clear operator checklist and return structured reasons.
Client-side filtering only.	Add query parameters on backend.
Large PDF exports in browser.	Server-side PDF generation for big datasets, date-range limits.

Chapter 04: System Analysis & Design

4.1 Functional & Non-functional Requirements

The following table 4 shows the functional requirements and table 5 shows non-functional requirements.

Table 4. Functional Requirements

Requirement	Priority
User Authentication & Role Routing	High
AI Complaint Priority Classification	High
Role-Based Dashboard Access	High
24-hour Digital Visitor Management System	High
Parcel Tracking and Notification	High
Online Billing System	High
Inter-vehicle Booking System	Medium
Notification System	Medium
User Feedback System	Low
PDF Download	Low

Table 5. Non-functional Requirements

Requirement	Priority
Performance	High
User-friendly Design	High
Scalability	Medium
Maintainability	Medium
Integrity and Safety	Low

4.2 Use Case Diagram



Figure 4. Use Case Diagram

The Havelock Smart Resident Portal is a comprehensive web-based platform for handling smart apartments and gate communities. The diagram shows four actors and their interactions with the system. Residents can manage their dashboard, create digital visitor passes, view parcels, request vehicle bookings, pay bills, submit maintenance complaints, and manage notifications. Admins oversee resident accounts, handle complaints, manage billing, generate reports, and configure system settings. Reception registers parcels, look up resident information, and handle front-desk operations. Security staff verify visitors at the gate, manage gate workflows, and conduct vehicle checks, ensuring community safety and access control.

4.3 System Architecture

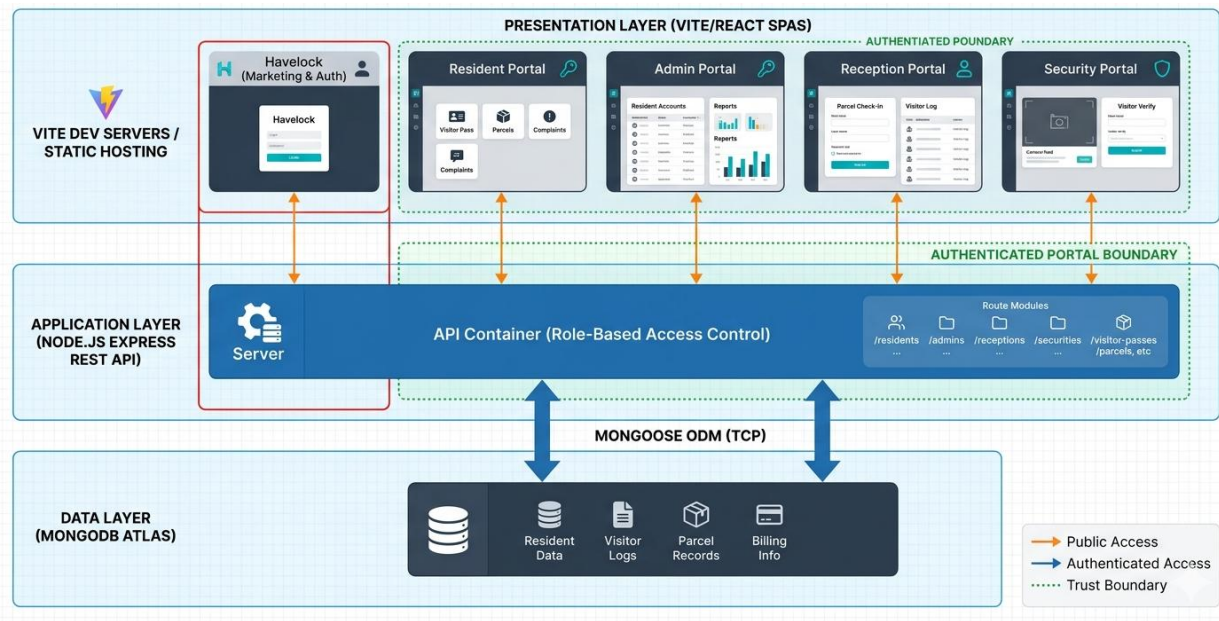


Figure 5. System Architecture

Havelock Smart Resident Portal is basically a role-based, separated ecosystem. This system is a “constellation” of five specialized applications. This make sure that a resident won’t view administrative back-office records and a security guard at a gate won’t be overwhelmed by billing charts. Each user interacts with a version of the system that is perfectly adapted to their physical environment.

1. The Presentation Layer

The frontend part of the system is develop using React and Vite, make sure the interface is extremely-fast and responsive. The public entry (Havelock) acts as the “lobby” where users sign in.

2. The Application Layer

The Node.js and Express API acts as the “brain” of the operation. Its sites in the middle, listening for requirements from the five portals.

- **Security and Validation:** When a Resident creates a pass, the API doesn’t just save it. It calculates the 24-hour expiration and make sure the pass is cryptographically valid before the security portal can see it.
- **The Bridge:** It translates human actions into structured data updates. It manages the heavy lifting – processing profile photos, calculating bills, and handling the logic of who can see it.

3. The Data Layer

All community life is stored within MongoDB. MongoDB uses flexible, document-based structure. This is perfect for a community management system because “data” varies greatly a “Vehicle booking” is just a collection of times and dates, while a “Complaint” may have a picture and a long-written description. Mongoose acts as the strict librarian here, make sure that even though the database is flexible, the information remains organized, searchable, and secure.

In summary, Havelock Smart Resident Portal isn't just a database with a website, it is a synchronized multi-user platform designed to turn the chaotic logistics of community living into a seamless, automated flow of information.

4.4 ER Diagram

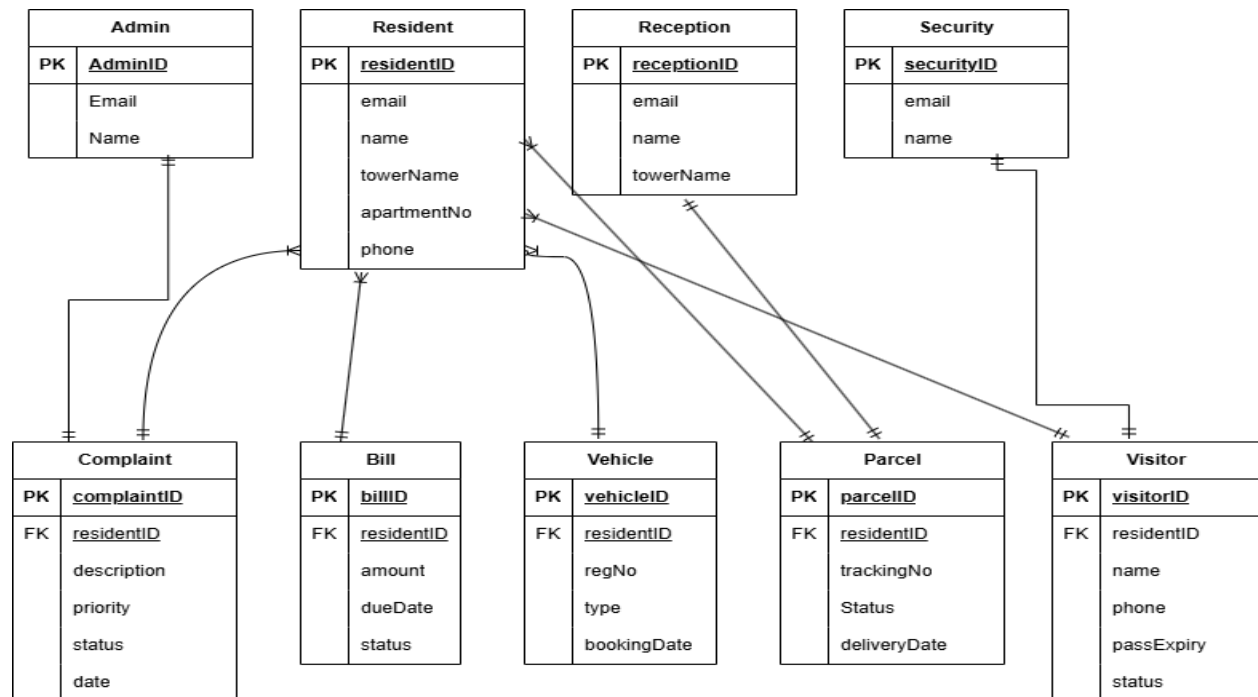


Figure 6. ER Diagram

This diagram illustrates an extensive apartment management system designed to help communication between staff and residents.

The focal point of the system on the Resident entity, which links to several functional modules. These modules including managing financial commitments through bills, registering personal vehicles, receiving parcels and hosting visitors. Operational oversight is split among specialized staff:

- **Admins** manage resident **Complaints**.
- **Reception** handles incoming **Parcels**.
- **Security** staff perform final checks on **Visitors**.

Each entity includes specific attributes like IDs, contact details, and status trackers to make sure data integrity and efficient residential workflow management.

4.5 Class Diagram

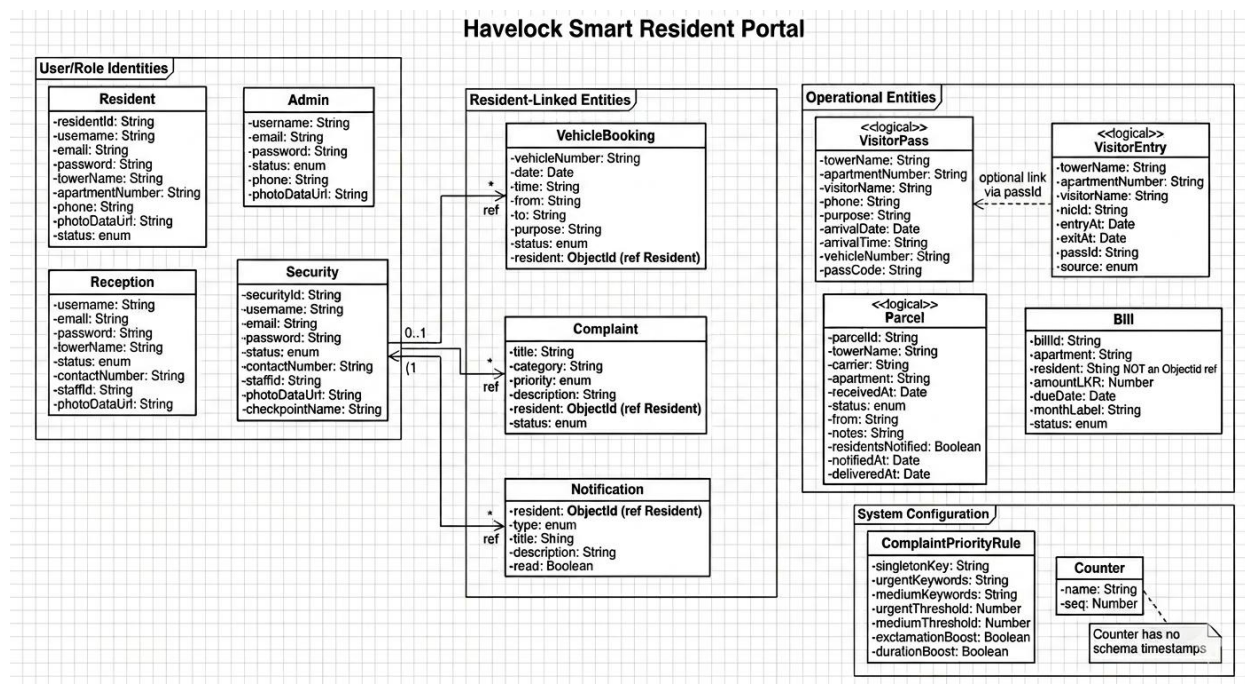


Figure 7. Class Diagram

The Havelock Smart Resident Portal's framework is created on a role-based, fragmented platform that breaks the presentation layer into five individual portals. This modular model makes sure that each user group – Residents, Admin, Reception, Security and the Public – operates within a unique interface designed to meet their specific operational requirements. The method ensures that the business logic for necessary gate actions stays apart from usual resident services while handling a unified UI consistency with the use of a monorepo structure with React and Vite. By limiting each platform's scope to its essential functional domain, this distinction improves security and reduces the mental impact on users.

The centralized management system at the application level is a Node.js and Express API that handles the conversion between user actions and data persistence. The backend is set up around domain-specific route groups that manage complex logic, like processing high-limit profile photo uploads or validating the 24-hour visitor passes expiration. The API ensures that only enabled access reach the data layer by enforcing role-based access control (RBAC) through its robust middle role.

MongoDB and Mongoose ODM, which have been chosen specifically for the ability they have to manage the varied and dynamic schemas of community management, are implemented in the data layer. The purposeful denormalization of operational records, like visitor entries and parcels logs, was a crucial architectural approach. The system maintains a reliable chronological audit trail even if a resident's account is eventually modified or deleted since apartment and tower details are stored as permanent strings rather than dynamic references. This ensures that the physical history of the gated community is separated from its residents.

A configuration-driven approach to non-functional requirements significantly maintains the system. Data clashes during periods of high traffic, such as monthly billing cycles, are prevented by integrating a single counter entity for atomic sequence generation. In addition,

the architecture is simple to scale from a local development environment to a production-level deployment employing HTTPS and secure CORS regulations thanks to the usage of environmental-based setups. This comprehensive design ensures the Havelock Smart Resident Portal is a coordinated digital nervous system that can handle the complex logistics of contemporary gated living, rather than just a collection of devices.

4.6 UI/UX Design

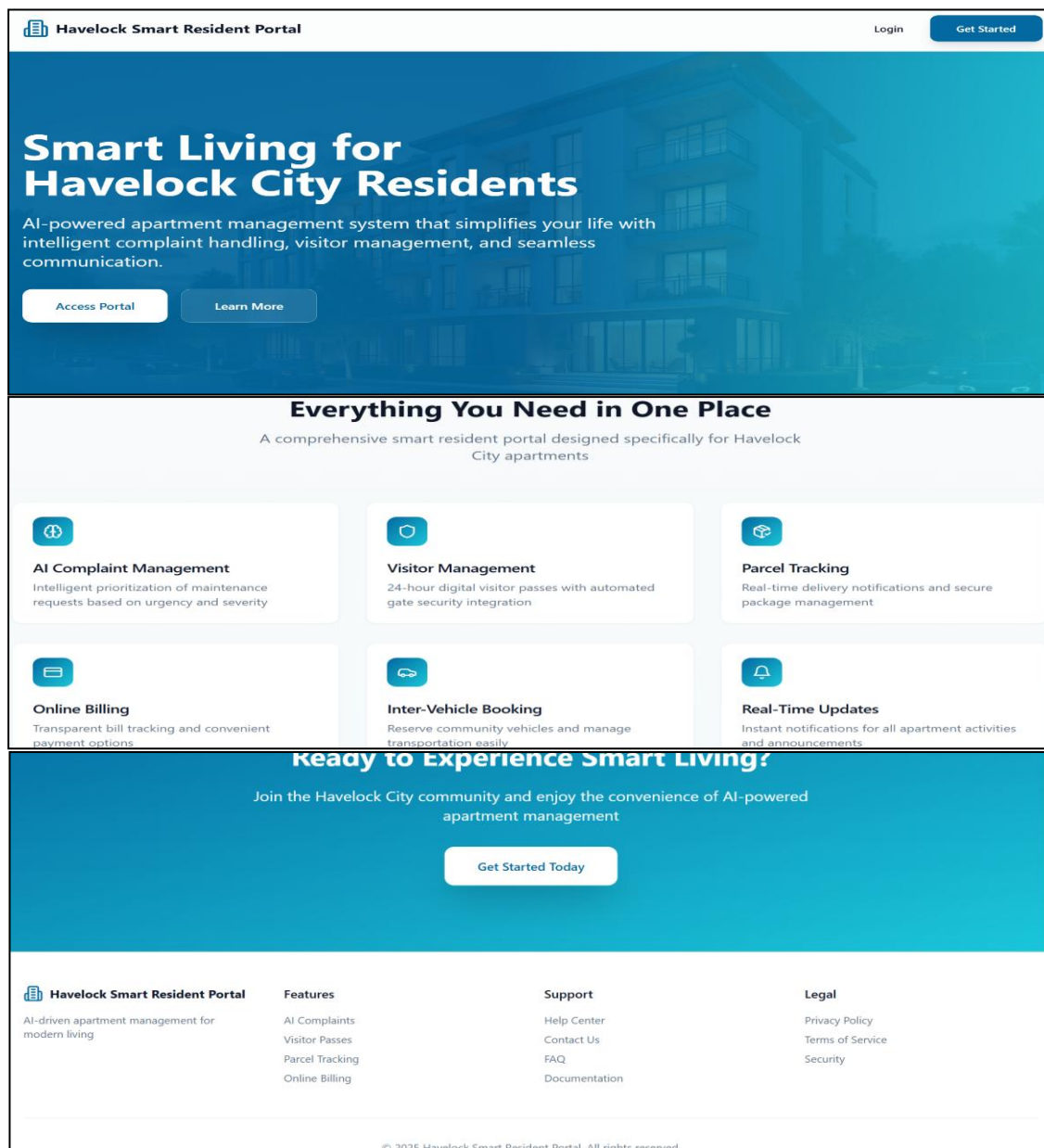


Figure 8. Home Page

The Havelock landing interface serves as the public entry point to the platform. It introduces the project vision, highlights core apartment-management capabilities, and provides clear call-to-action elements that guides users the portal access. The interface designed for first-time users, offering a concise product overview and intuitive navigation before authentication.

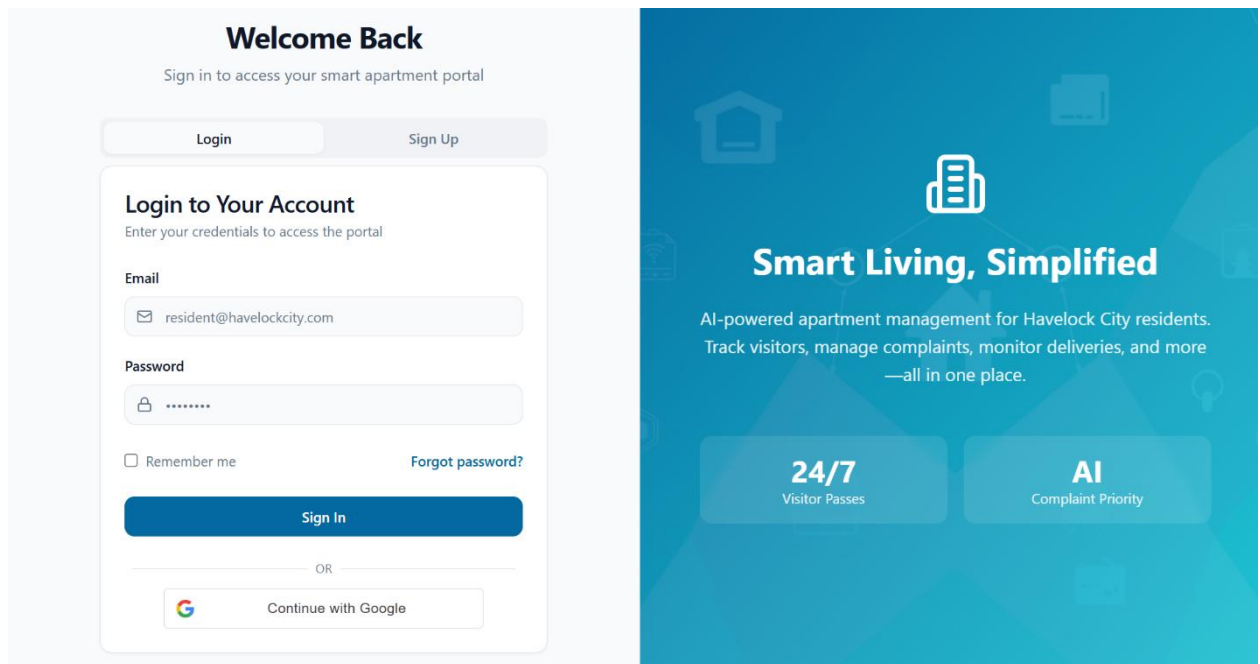


Figure 9. Authentication Interface

The authentication interface provides a centralized login and registration experience for multiple user roles, including Resident, Admin, Reception and Security. It supports credential-based authentication and resident Google sign-in integration. This interface is responsible for initiating role-based direction to the appropriate portal and acts as the primary security gateway to the system.

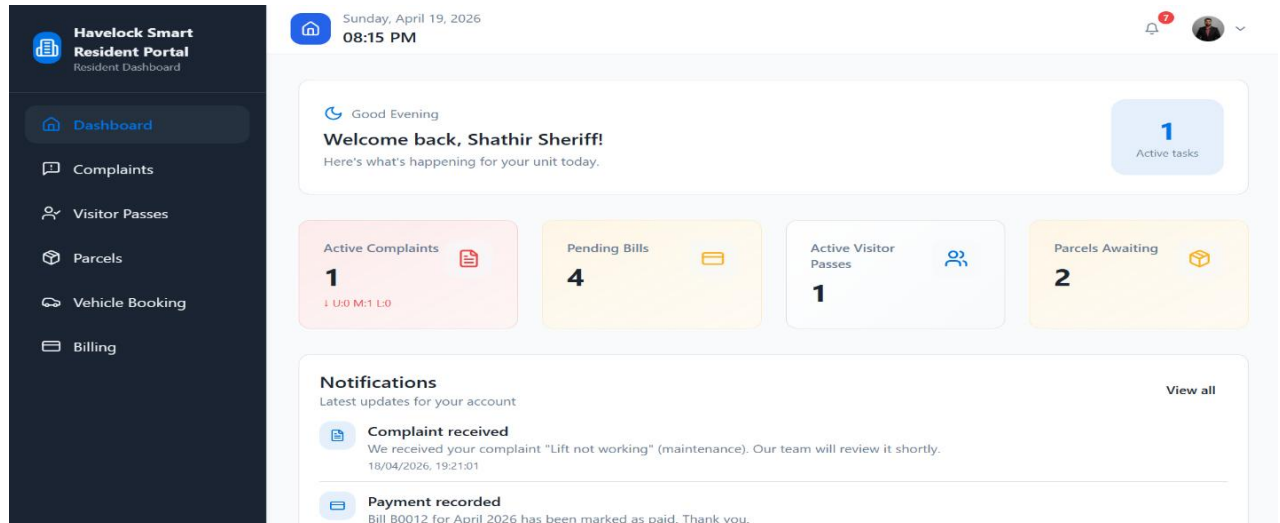


Figure 10. Resident Dashboard

The Resident dashboard interface provides a high-level overview of resident-specific activities, combining summary widgets, quick actions, and personalized contextual information. It is the centring monitoring screen for daily resident interactions.

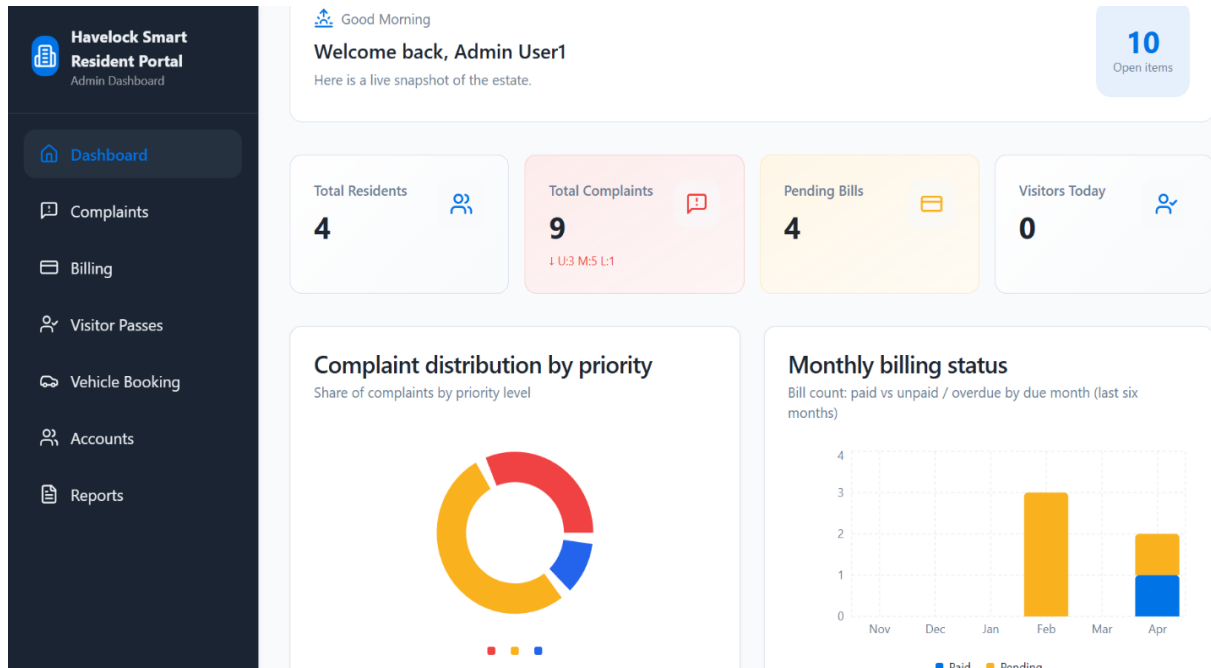


Figure 11. Admin Dashboard

The admin dashboard interface provides operational visibility across core modules through summary cards and analytical indicators. It helps administrators quickly evaluate system-wide activity and identify priority areas.

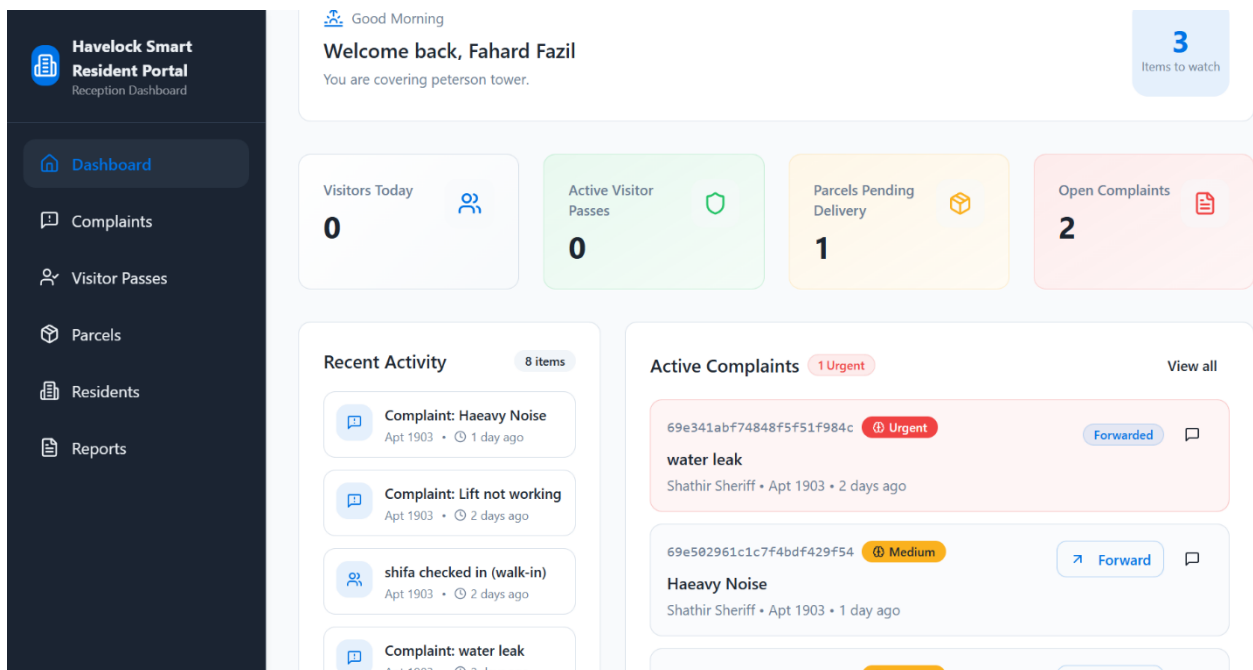


Figure 12. Reception Dashboard

The figure 12 reception dashboard interface is optimized for front-desk workflows by presenting tower-oriented operational context and quick access to key modules. It supports high-frequency tasks performed by reception staff.

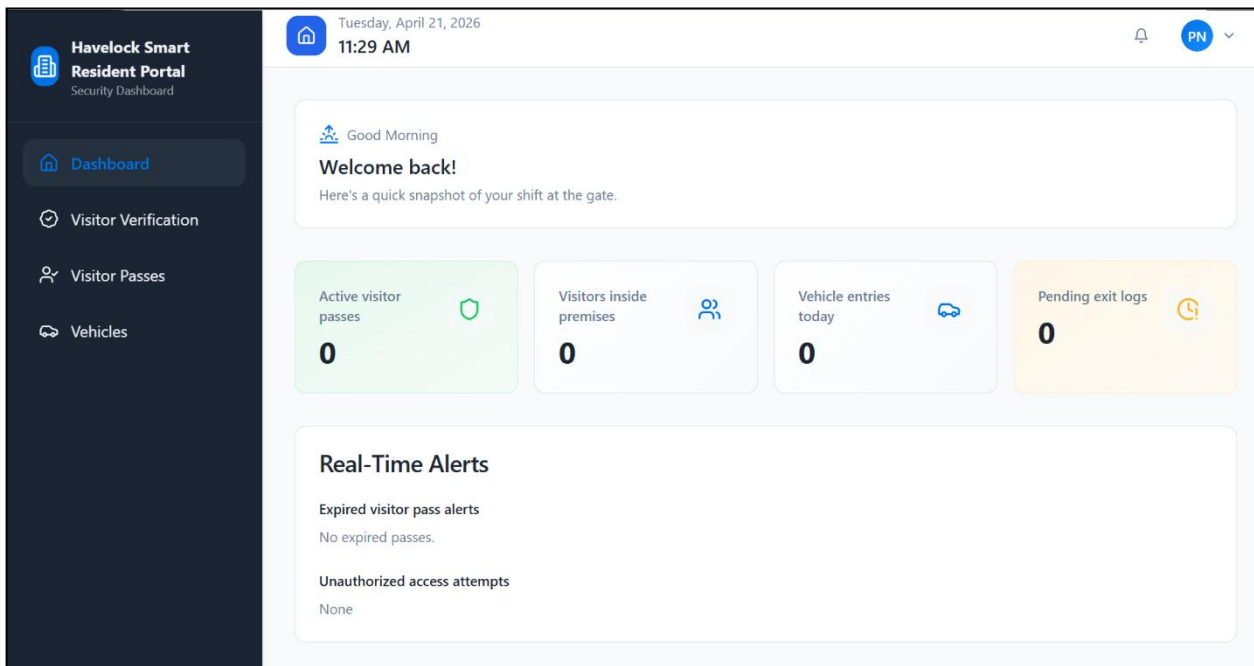


Figure 13. Security Dashboard

The security dashboard interface presents gate-focused daily activity and alert context, enabling security staff to monitor high-priority operational events efficiently.

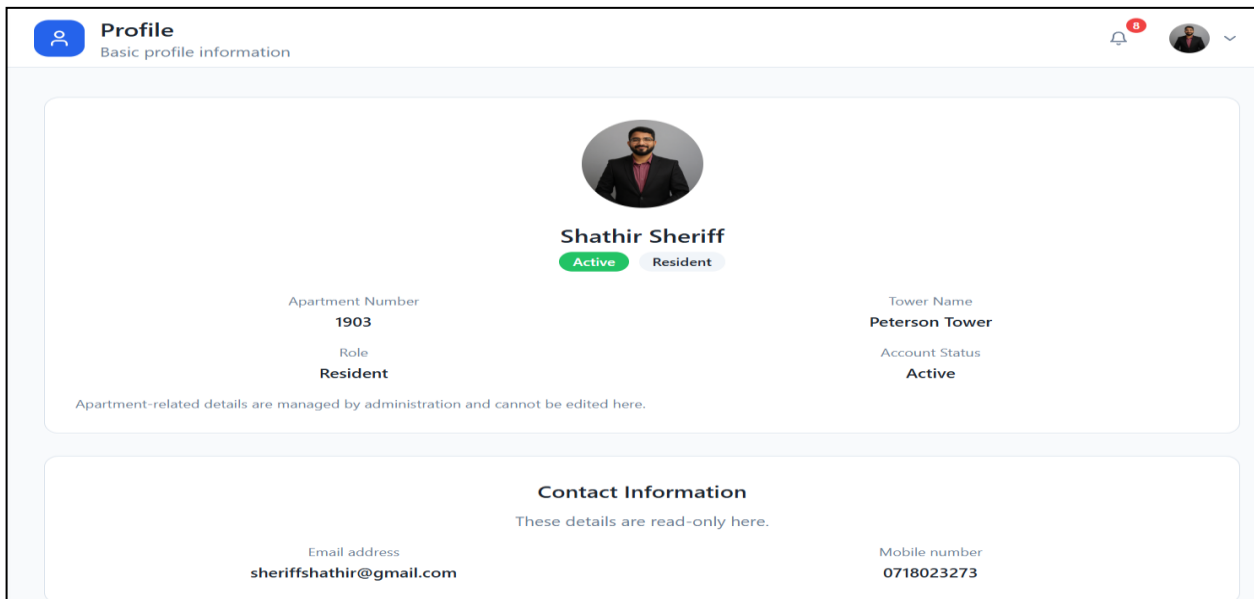


Figure 14. User Profile

The above image shows a structured view of resident identity and associated information. The other users (Admin, Reception, Security) have the same structure view of their identity and associated information. The staff users profile page they can see a staff id. And in the all-users profile page shows the account status and their role.

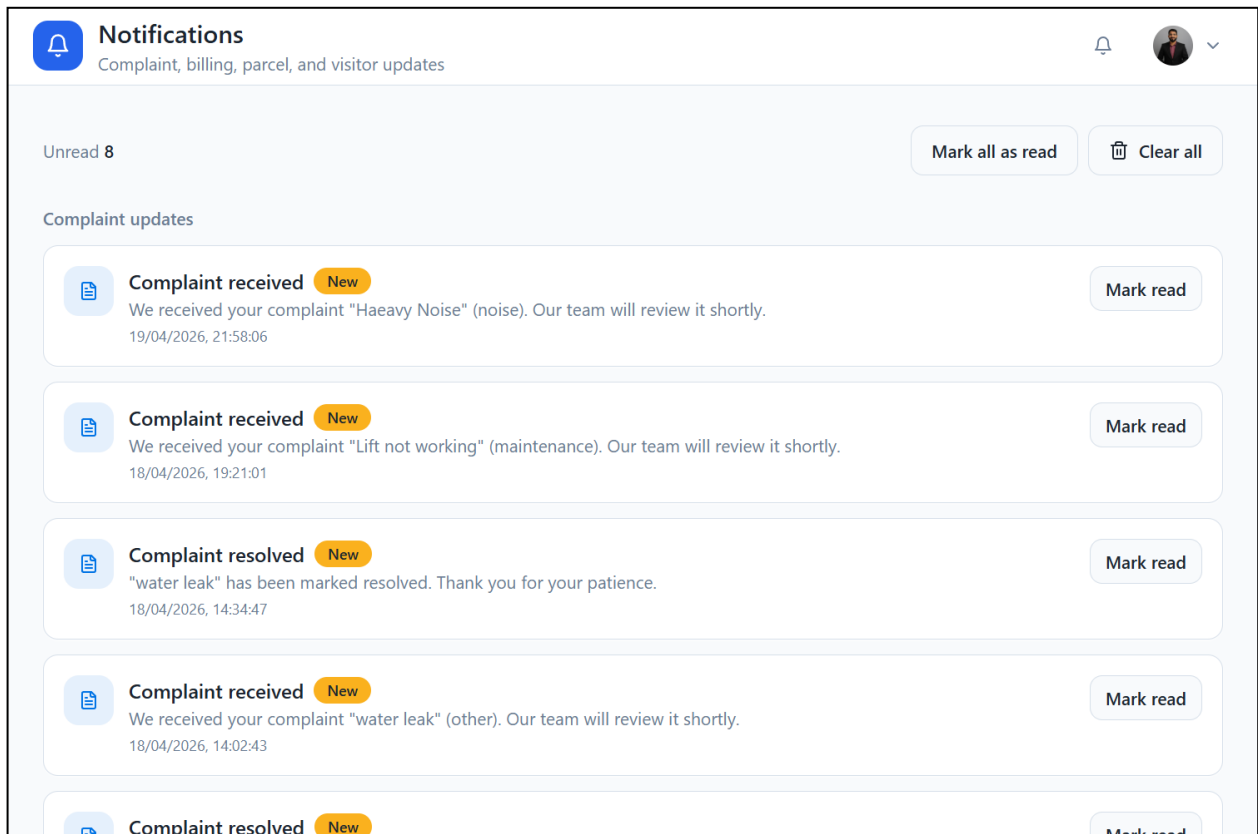


Figure 15. Notification Interface

The above image is to get notification alerts for recent activities which related to the user. The notification alerts are going separately to different users and different roles. Users' role like Resident will get a notification alert which he has any bill payments to pay or a parcel received at the reception desk. So, this is very useful to get quick actions on their daily tasks inside the Havelock City.

4.7 Network Diagram

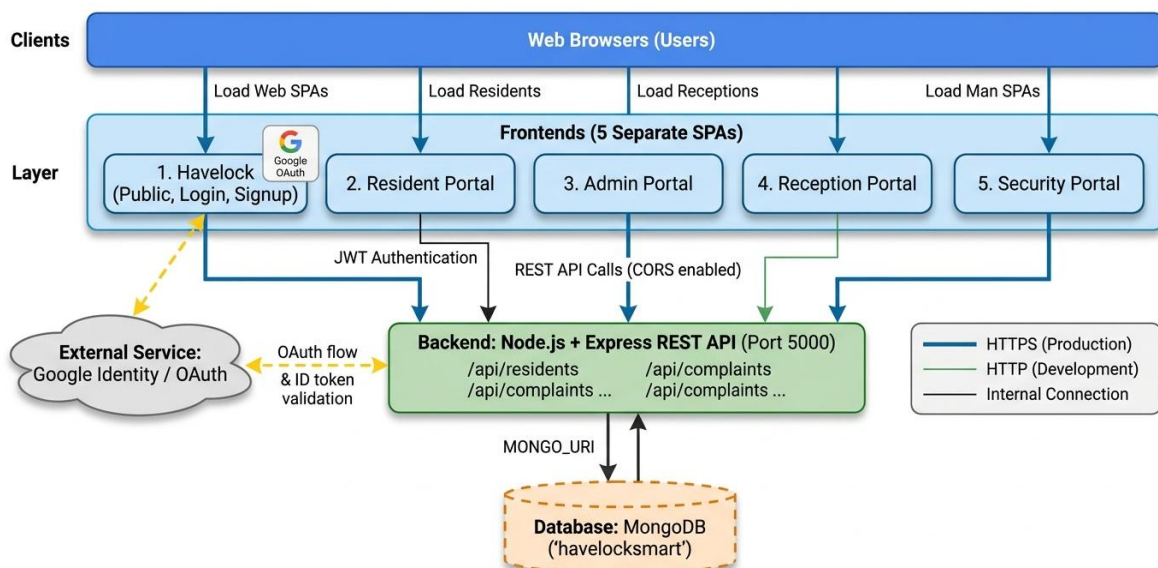


Figure 16. Network Diagram

The above diagram illustrates the network diagram of the Havelock Smart Resident Portal, organized into a four-tier structure. It demonstrates how a user's request moves from the browser to the data storage after going through various security levels.

1. The Client Layer (Top)

The blue bar shows the end-user environment (chrome). This is where the physical interaction happens. The arrow pointing down shows the browser loading one of the five specific Single Page Applications (SPAs) based on the user's role.

2. Frontend Layer (The Five SPAs)

Every box shows a separate React application.

- **Havelock:** Acts as the entry point. Notice the Google OAuth icon here; this is where the external authentication handshake begins.
- **Portals:** These are the functional hubs for different user types.
- **Authentication Flow:** The dash yellow line shows the "OAuth Flow". The browser talks to Google, gets a token, and passes it to the backend to exchange it for a JWT, which the frontend then stores to keep the user (Resident) logged in.

3. Backend Layer (The Engine)

The green box shows the Node.js + Express API.

- **CORS** – The diagram notes "CORS" enabled," which is a security configuration allowing different frontend URLs to talk to API URL without being blocked by the browser.
- **Route Groups** – It lists internal endpoints like "api/residents" and "api/complaints" showing how the backend organizes the different business logic.

4. Data Layer (Bottom)

The orange cylinder shows MongoDB.

- The connection is labelled "MONGO_URI" which is the environment variable used to bridge the backend and the database.
- The double-headed arrows indicate a bi-directional flow: The API sends data to be saved and receive data back.

5. Connectivity & Security

- **Blue lines:** Represent HTTPS.
- **Green lines:** Represent HTTP.
- **Internal connections:** show how backend respond to the database or verifies tokens behind the scenes.

Chapter 05: System Implementation

5.1 Frontend Development

The user-friendly interface is delivered as a monorepo of five independent single-page applications (SPAs), each implemented with Vite, React and TypeScript. The frontends share a common technical approach: Tailwind CSS for styling, shadcn/ui for accessible components, React Router v6 for navigation, and TanStack Query where server state benefits from caching and background refresh. Applications communicate with a separate Node.js/Express REST API using environment-based base URLs.

The five applications are role-specific: Havelock (marketing and unified authentication entry), Resident, Admin, Reception, and Security. This separation keeps bundles focused, allows independent deployment, and maps cleanly to user responsibilities.

5.1.1 Technology Stack

Table 6. Frontend Tech Stack

Concern	Technology	Role
Build / dev server	Vite 5	Fast HMR, ES modules, production bundling.
UI library	React 18	Component model, concurrent features.
Language	TypeScript	Static typing for components & API shapes.
Styling	Tailwind CSS	Utility-first layout & design tokens.
Component kit	Shadcn/ui + Radix	Accessible dialogs, menus, forms, tooltips etc.
Routing	React-router-dom 6	Declarative routes, layout pages, URL-driven state.
Server state	TanStack Query 5	Queries/mutations, caching, loading & error states.
Forms	React-hook-form + Zod	Validation and controlled forms.
Icons	lucide-react	Consistent iconography across apps.
Charts	Recharts	Dashboard chart where used.
OAuth (Havelock)	@react-oauth/google	Google sign-in for residents.
Notifications UI	Sonner/toast primitives	User feedback for actions.

5.1.2 Monorepo Layout & Responsibilities

Table 7. Monorepo Responsibilities

Application	Primary users	Main UX focus
Havelock	All roles (entry)	Marketing hero, sign-up, multi-role password login, resident Google OAuth, redirect handoff.
Resident	Tenants	Data scoped to unit; session bootstrap; JWT for API authorization when present.
Admin	Management	Cross-resident views, charts, accounts, complaint priority rules.
Reception	Front desk	Visitor entries, parcels, tower-scoped lists.
Security	Guards	Verification, vehicles, filtered notifications.

5.1.3 Routing & Global Providers

- **React Router** defines paths per app.
- **TanStack Query** wraps the app for “useQuery / useMutation”.
- **TooltipProvider** and toast / sonner components support non-blocking feedback.

Session handoff: Havelock redirect with Query parameters. Target apps persist localStorage, clean the URL, and dispatch events for data refresh.

5.2 Backend Development

The backend is a RESTful HTTP API that centralizes data and business rules for multiple client applications (Resident, Admin, Reception, Security). It is implemented in Node.js using the Express web framework and persists application state in MongoDB via Mongoose. The API exposes resource-oriented routes under the “/api” prefix and is designed to be consumed by separate frontend applications over CORS enabled HTTP requests.

5.2.1 Technology Stack

Table 8. Backend Technology Stack

Layer	Technology	Role
Runtime	Node.js	Server execution environment
HTTP	Express (v5)	Routing, middleware pipeline, JSON body parsing.
Database	MongoDB	Document store for all domain entities.
ODM	Mongoose	Schema definitions, validation, queries
Security	bcryptjs	Password hashing for credential-based accounts.
Security	jsonwebtoken	JWT issuance for Google-authenticated residents.
Security	google-auth-library	Verification of Google id tokens.

Cross-origin	cors	Browser cross-origin access for multiple frontend origins.
Configuration	dotenv	Environment-driven configuration (MONGO_URI, PORT, JWT and Google settings).
Observability	morgan	HTTP request logging.

5.2.2 High Level Architecture

The implementation follows a conventional three-layer structure.

- 1. Routes** – Map URL paths and HTTP methods to controller functions. Route files are mounted in “server.js” under namespace prefixes.
- 2. Controllers** – Implement request handling validate input, call mongoose models, apply domain rules and send HTTP responses.
- 3. Models** – Define Mongoose schemas and encapsulate persistence for entities.

A small library module (backend/src/lib/complaintPriorityFromRules.js) implements rule-based complaint prioritization aligned with admin settings stored in database.

Database connectivity is initialized in “backend/src/config/db.js”: the system connects to MongoDB using “MONGO_URI” and uses the database name “havelocksmart”.

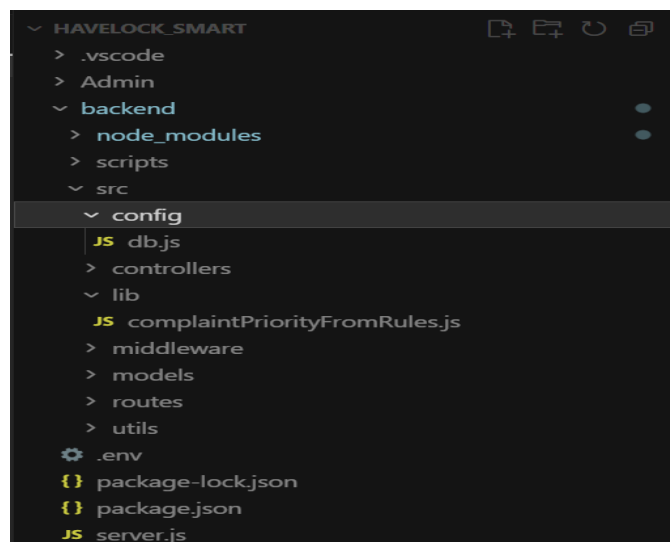


Figure 17. Backend Folder Structure

5.2.3 Data Models & Identifiers

Core collections include residents, admins, receptions, securities, visitorpasses, vehiclebookings, bills, complaints, parcels, visitorentries, notifications and complaintpriorityrules. A counter collection supports human-readable sequential identifiers issues at signup or via maintenance scripts. Relationships use Mongoose references where appropriate.

5.2.4 Domain Logic Highlights

1. Complaint priority.

When complaints are created or listed, priority (urgent, medium, low) can be computed using admin-configurable rules persisted in MongoDB. The algorithm combines:

- Keyword lists for urgent medium severity.
- Numeric thresholds and scoring.
- Optional boosts.

2. Notifications & Side effects.

Controllers for parcels and bills can create notification documents for affected residents when business rules match. This ties operational events to the resident notification center and unread counts.

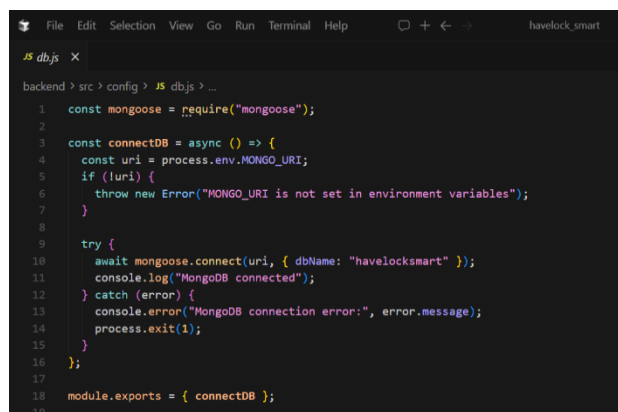
3. Authentication Models.

- Password-based flows use bcrypt hashing for stored credentials.
- Google sign-in for residents verifies the Google id token, make sures the email is verified, resolves the resident record, and returns a JWT.

5.3 Database Integration

The API persists domain data in MongoDB using the Mongoose ODM. One connection is opened at startup; entities are schemas/models; Express controllers call Mongoose APIs. The database name “havelocksmart” is set in the code; the cluster URI is MONGO_URI in .env.

• Connection Setup



```
1 const mongoose = require("mongoose");
2
3 const connectDB = async () => {
4   const uri = process.env.MONGO_URI;
5   if (!uri) {
6     throw new Error("MONGO_URI is not set in environment variables");
7   }
8
9   try {
10    await mongoose.connect(uri, { dbName: "havelocksmart" });
11    console.log("MongoDB connected");
12  } catch (error) {
13    console.error("MongoDB connection error:", error.message);
14    process.exit(1);
15  }
16 };
17
18 module.exports = { connectDB };
19
```

Evidence: backend/server.js

```
const { connectDB } = require("../src/config/db");
require("dotenv").config();
// Database Connection
connectDB();
```

Figure 18. Connection Setup

- **Models Barrel**

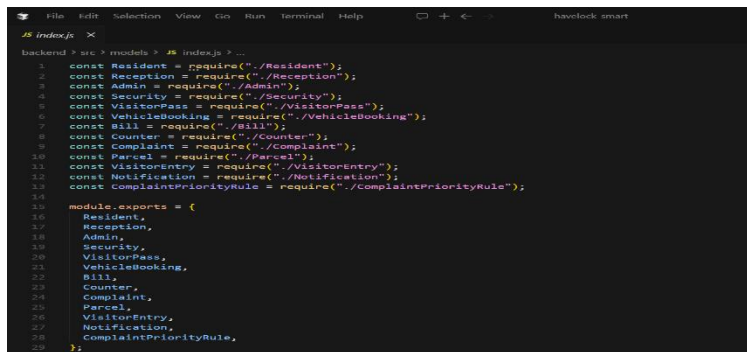


Figure 19. Models Barrel

- **Schema, validation & timestamps**

Evidence: backend/src/models/Resident.js

```

const residentSchema = new mongoose.Schema(
  {
    residentId: { type: String, unique: true, sparse: true, trim: true },
    username: { type: String, required: true, unique: true, trim: true, minlength: 3 },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    password: { type: String, required: true, minlength: 8 },
    towerName: { type: String, required: true, trim: true },
    apartmentNumber: { type: String, required: true, trim: true },
    phone: { type: String, trim: true },
    photoDataUrl: { type: String, default: null },
    status: { type: String, enum: ["active", "blocked"], default: "active" },
  },
  { timestamps: true }
);

```

- **Relationships & Collection name**

Evidence: backend/src/models/Complaints.js

```

resident: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "Resident",
  default: null,
},
status: {
  type: String,
  enum: ["pending", "in-progress", "resolved"],
  default: "pending",
},
},
{
  timestamps: true,
  collection: "complaints",
}
);

```

- **Controller Integration**

Evidence: backend/src/controllers/residentController.js

```
const { Resident } = require("../models");

const getById = async (req, res) => {
  const { id } = req.params;
  const resident = await Resident.findById(
    id,
    "_id residentId username email towerName apartmentNumber phone photoDataUrl status createdAt"
  );
  if (!resident) {
    return res.status(404).json({ message: "Resident not found." });
  }
  return res.json({ /* mapped fields */ });
};

// Signup: duplicate check + create
const existing = await Resident.findOne({ $or: [{ username }, { email }] });
// ...
const resident = await Resident.create({
  username, email, password: hashedPassword, towerName, apartmentNumber,
  phone, status: status === "blocked" ? "blocked" : "active", photoDataUrl,
});
```

Controllers use “findById, findOne, create”, and map documents to JSON. This is running integration layer between HTTP and MongoDB.

5.4 AI Module Implementation

5.4.1 System Overview: Simple AI Rule-Based Logic Method

For this system, Researcher didn’t use a trained machine model whether researcher mentioned it as “AI assisted” in system user interface, it is technically a simple rule-based logic technique that assigns each maintenance priority label as Urgent, Medium, Low by identifying the keyword.

How it works?

The system checks incoming maintenance requests by analyzing the category, title and description. After that, it calculates a score based on unique inputs and matches that score with one of three priority levels: Urgent, Medium or Low.

5.4.2 The Logic

The intelligence of the module lives in a scoring algorithm that weighs several factors:

- **Keyword Scoring:** The system scans the text for specific “urgent” or “medium” keywords defined by admins.

- **Linguistic Cues:** It looks for text patterns like exclamation marks or duration phrases (e.g., “for three days”) to increase the score.
- **Category Boosts:** Critical infrastructure categories like security, water, electricity, and plumbing receive automatic weighting increases.
- **Threshold Mapping:** Finally, the total score is compared against the preset thresholds (e.g.: a score of 3 or higher becomes “Urgent”).

5.4.3 Implementation & Data Flow

The module is integrated across the full stack to make sure transparency for both residents and staff.

- **The Admins’ Role (Configuration)**
In Admin’s settings page allows admin to customize the system. There is a single “master” set of rules for the entire application since these rules are maintained in the database as a singleton document.

Figure 20. Admin's Security Page Complaint Priority Rules

- **The Resident’s Role (Live Feedback)**
The system provides live feedback of the recommend priority as a resident type a complaint. Because the resident could see the reason (e.g.; “detected keyword: water”) behind the system’s suggestion.

localhost:5173/complaints

Havelock Smart Resident Portal
Resident Dashboard

- Dashboard
- Complaints**
- Visitor Passes
- Parcels
- Vehicle Booking
- Billing

File New Complaint

Fill in the details below to submit your complaint

Complaint Title *

water leak

Category *

Water

Description *

There is a water leak

AI Feedback

AI Priority **Urgent**
Based on keywords: leak • category: water

Submit Complaint **Cancel**

Figure 21. The Live Feedback

Chapter 06: Testing & Evaluation

6.1 Introduction to Testing

During the testing phase of the Havelock Smart Resident Portal's development, the goal changed from developing functions to make sure the system performed well in practical situations. In addition to discovering issues, testing confirmed that residents, security staff, and management could use the system without trouble.

Initially, testing start with basic feature such as role-based access, user registration, and login. All user types, such as residents, admins, receptions and security staff, were able to safely access the platform due to these initial tests. More advanced features, like AI-based complaint prioritizing, visitor pass generation, parcel tracking, and online billing, were tried as development proceeded. More attention was taken to make sure that visitor approvals could be processed quickly and urgent issues were appropriately classified.

The system was tested regularly during implementation as well as after all modules were integrated. The frontend, backend, and database all worked correctly because of integration testing. Additionally, real-time functionalities like updates and notifications were carefully examined to ensure users get fast and accurate information.

This chapter provides the outcomes of the testing phase and highlights the improvements made based on the results. It contains evaluation results, test case tables, and system screenshots. These results are important for improving the final version of the system, ensuring that each user of the Havelock Smart Resident Portal has a reliable, user-friendly, and effective experience.

6.2 Testing Methodology

The validation process for the system followed a “bottom-up” approach. By approach individual functions for verification, the researcher ensured that core logic was sound before merging components into the broader system architecture.

Documentation for these focused on:

- Assigning specific IDs and sequence numbers to every test.
- Describing the exact behaviour or feature under review.
- Expected results.
- Achieved or not.
- Evidences.

6.3 Test Cases

6.3.1 User Authentication

Table 9. Test Case 01

Test Case: Signup & Login Pre-condition: User in Login Page			
TC#	Sequence#	Description	Expected results
01	1.1	Login with correct credentials.	Login successful and redirects to relevant user dashboard.

- Successful message when login was successful.

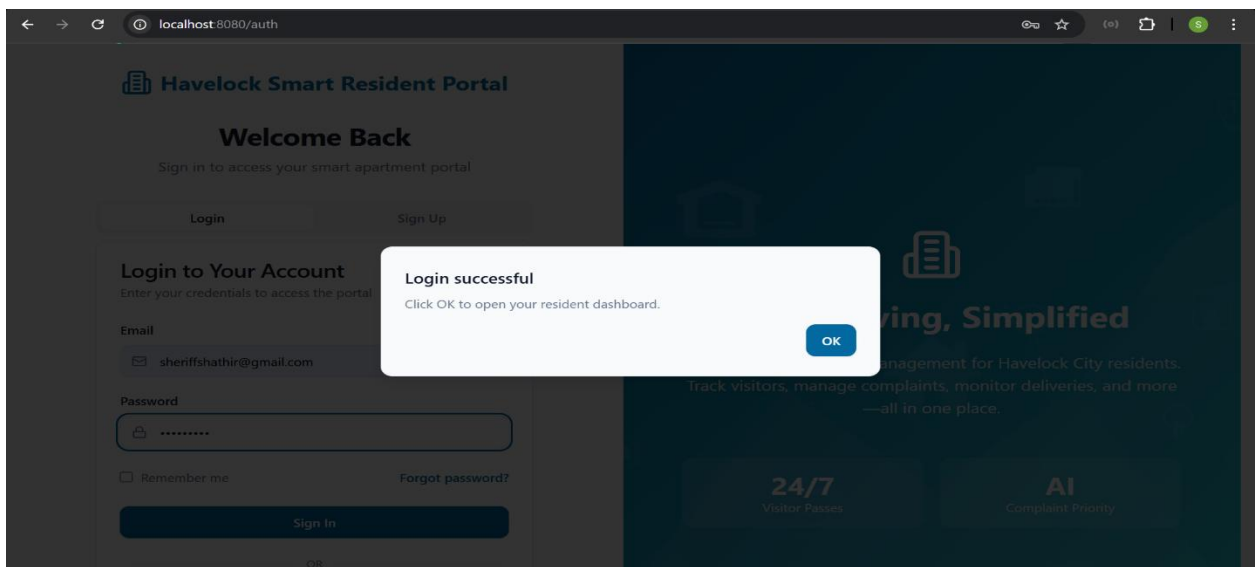


Figure 22. Evidence 01

- Redirects to relevant user dashboard page.

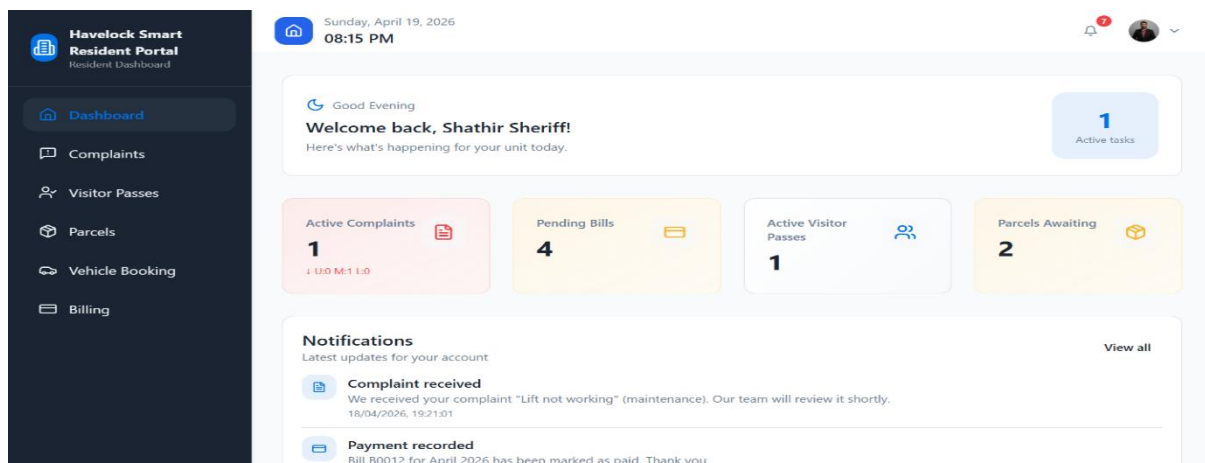


Figure 23. Evidence 02

02	1.2	Login with invalid credentials.	Display error message and still in login page.
----	-----	---------------------------------	--

- Error message and still on login page.

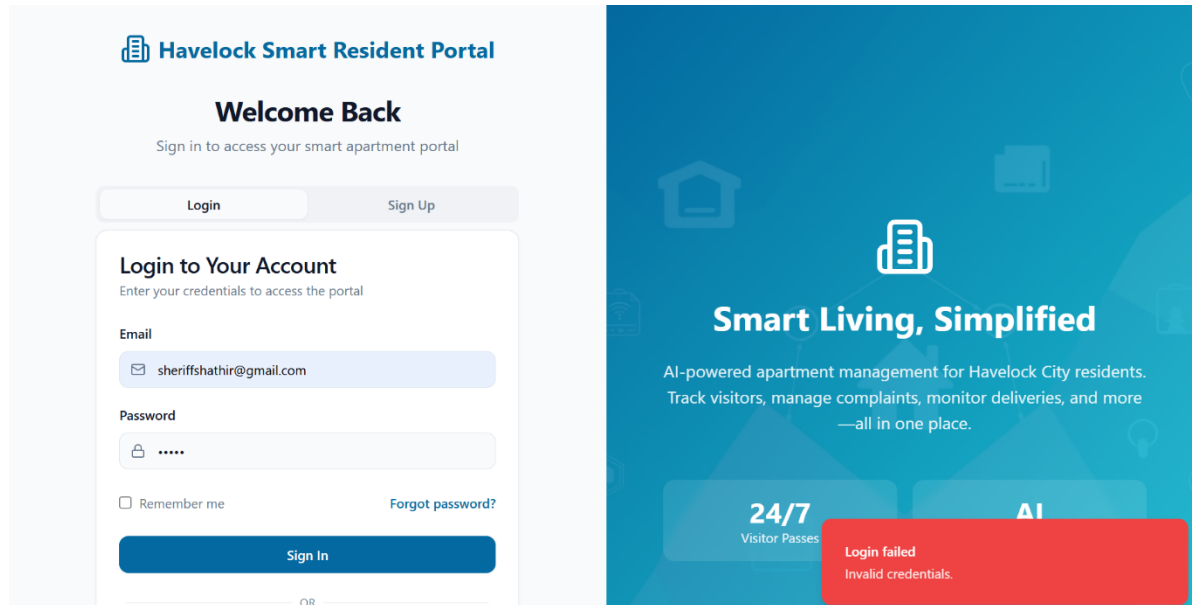


Figure 24. Evidence 03

03	1.3	Resident login with Registered Google account.	Login successful. A JWT session token was created for your account. Click ok to open your resident dashboard.
----	-----	--	---

- Select the Google account.

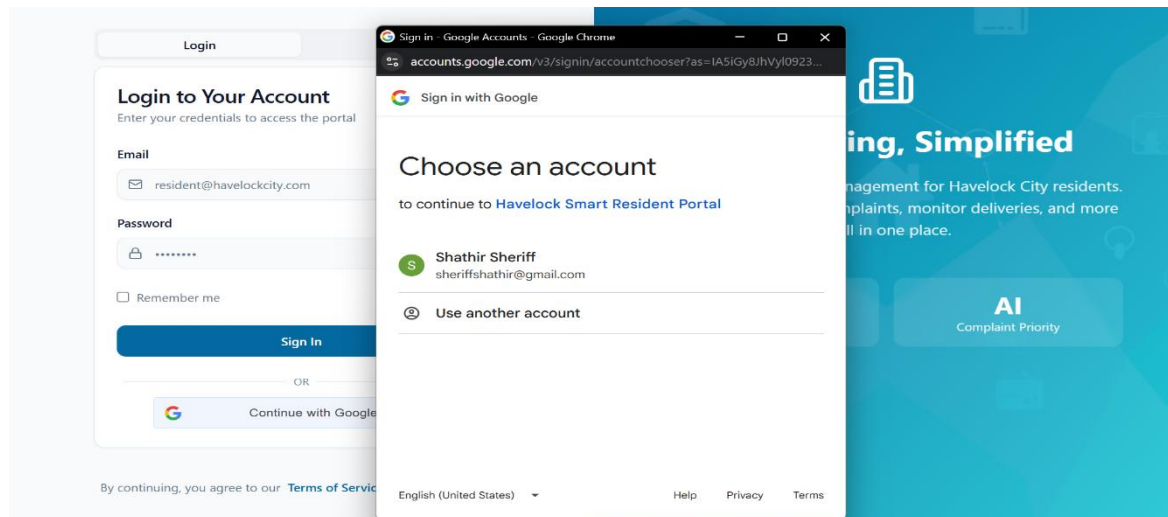


Figure 25. Evidence 04

- Successful message when login successful.

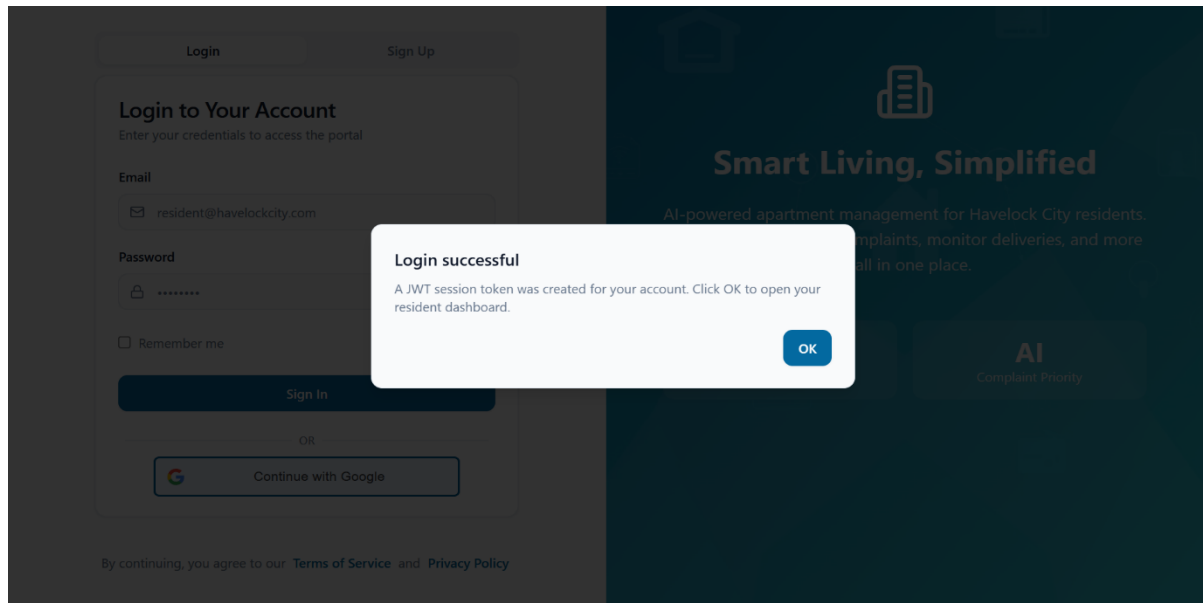


Figure 26. Evidence 05

04	1.4	Resident login with a Google account not registered in the site.	Google sign-in failed.
----	-----	--	------------------------

- Message appear and still in the login page.

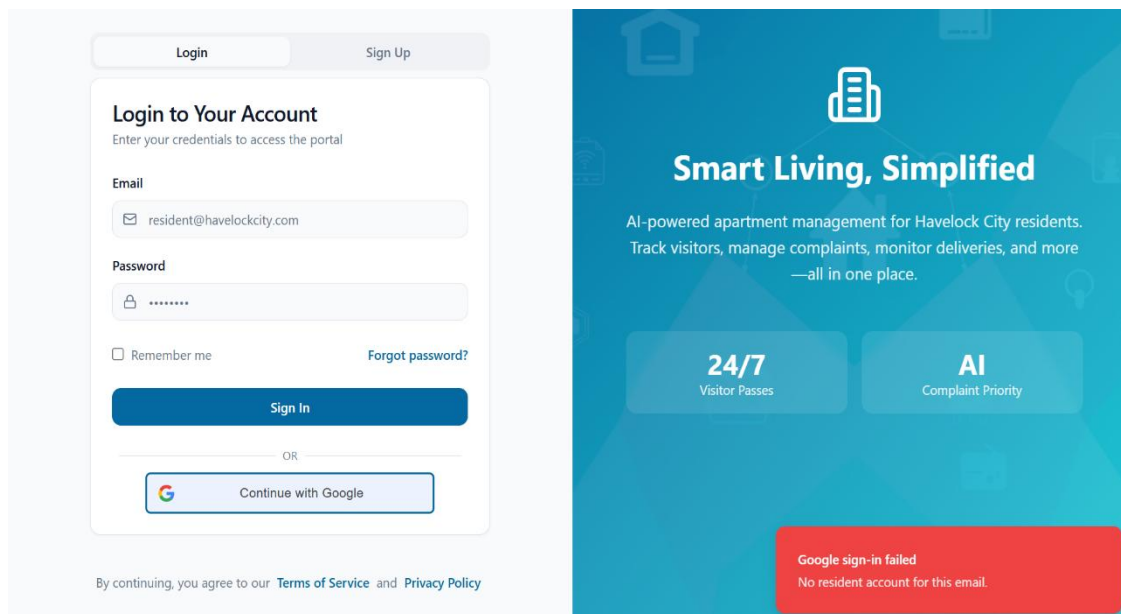


Figure 27. Evidence 06

Table 10. Test Case 02

Test Case: Logout Pre-condition: User in Dashboard page.			
TC#	Sequence#	Description	Expected results
05	2.1	User click logout button.	Message appear and after few seconds logout from the dashboard.

- Under the profile icon click the logout button.

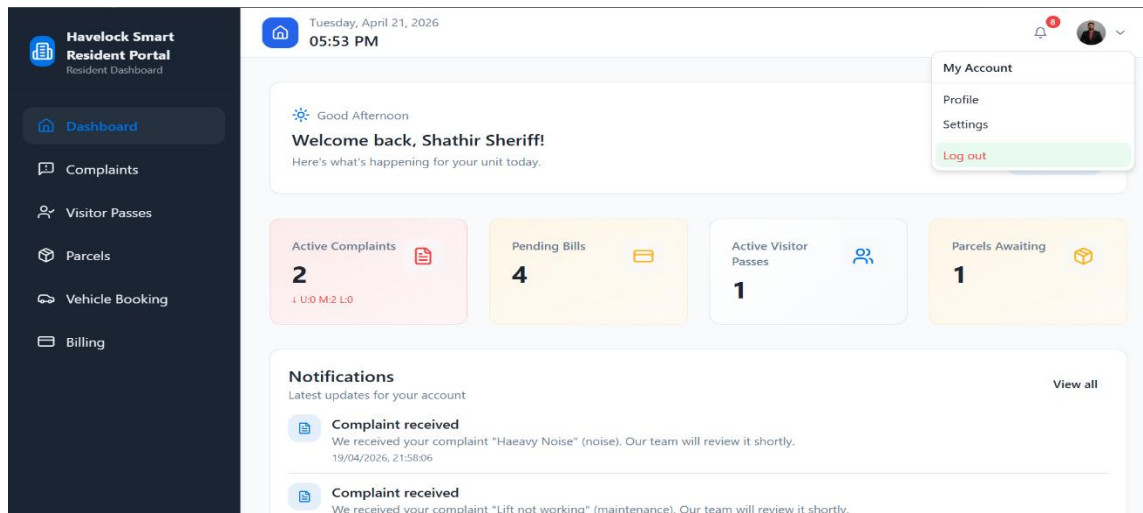


Figure 28. Evidence 07

- Success message appears.

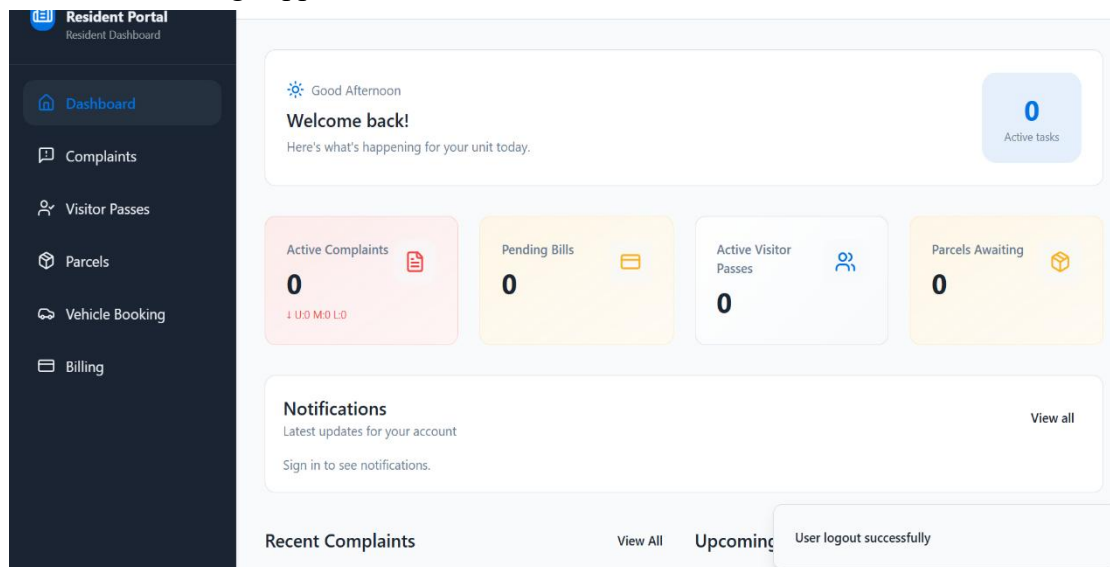


Figure 29. Evidence 08

6.3.2 Complaint Management

Table 11. Test Case 03

Test Case: Complaint Management - Resident

Pre-condition: Resident in Login Page

TC#	Sequence#	Description	Expected results
06	3.1	Submit the complaint without filling fields.	Message appear: Please fill in this field.

- Message appear and doesn't submit the complaint.

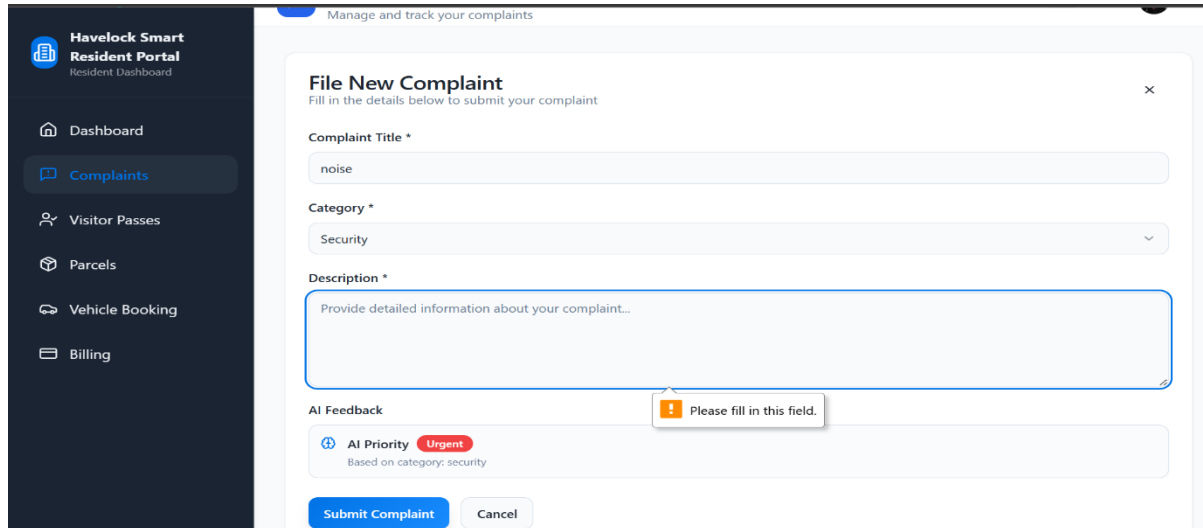


Figure 30. Evidence 09

07	3.2	Submit the complaint with complete filling all the fields.	Complaint Filed
----	-----	--	-----------------

- Stay on complaint page and see the success message of complaint filed.

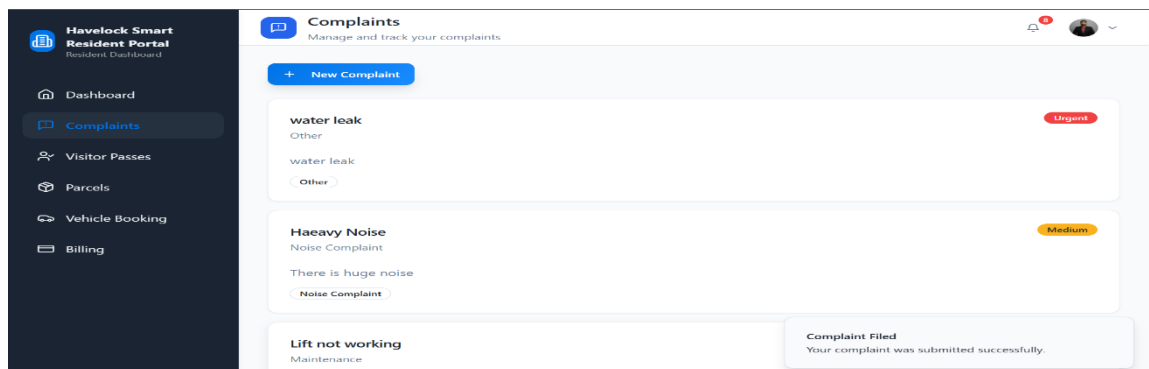


Figure 31. Evidence 10

08	3.3	Check the complaint priority.	Get live feedback for complaint priority.
----	-----	-------------------------------	---

Note: The Residents' complaints priority identifies from the keywords which match with the complaint priority rules form which handles by Admin.

- AI Feedback as **Urgent** from how complaint fills.

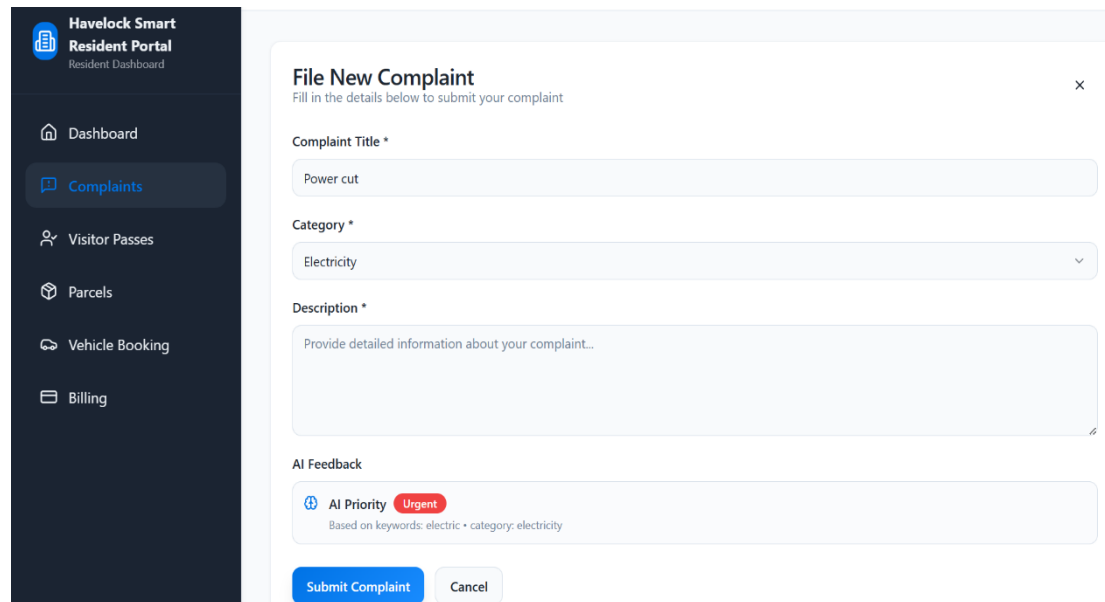


Figure 32. Evidence 11

- AI Feedback as **Medium** from how complaint fills.

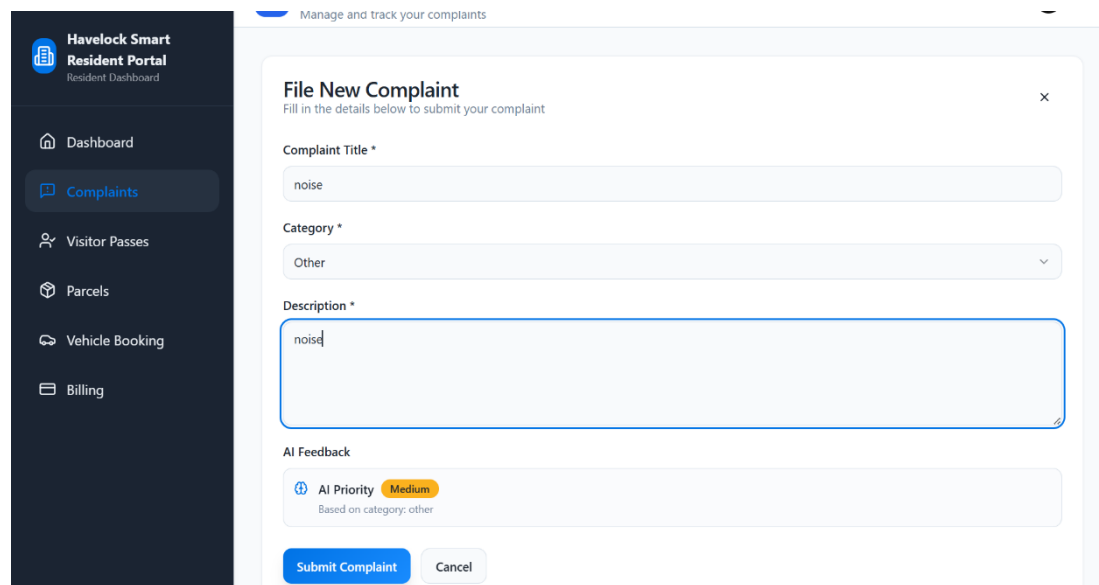


Figure 33. Evidence 12

- AI Feedback as **Low** from how complaint fills.

Havelock Smart Resident Portal
Resident Dashboard

- Dashboard
- Complaints**
- Visitor Passes
- Parcels
- Vehicle Booking
- Billing

File New Complaint

Fill in the details below to submit your complaint

Complaint Title *
Brief description of the issue

Category *
Other

Description *
Provide detailed information about your complaint...

AI Feedback
AI Priority: **Low**
Based on category: other

Submit Complaint **Cancel**

Figure 34. Evidence 13

Table 12. Test Case 04

Test Case: Complaint Management - Admin
Pre-condition: Admin in Complaint page.

TC#	Sequence#	Description	Expected results
09	4.1	Admin want to be able to view the complaint message.	Admin view the message.

- Admin view the complaint message.

Havelock Smart Resident Portal
Admin Dashboard

- Dashboard
- Complaints**
- Billing
- Visitor Passes
- Vehicle Booking
- Accounts
- Reports

Complaint Management

AI-powered complaint prioritization and tracking • Live data

Total Complaints: 1

Complaint CM0001 Shathir Sheriff • Apt 1903

Submitted on 2026-04-21 • Category: Electricity

Title
No electricity

Full description
Fan is not working

AI assessment: Based on keywords: electric • category: electricity

Created at
21/04/2026, 19:07:24

Updated at
21/04/2026, 19:17:48

Update status
In Progress

Update status **Assign staff** **Close complaint**

Figure 35. Evidence 14

10	4.2	Admin assigned a staff member for the complaint.	Admin assign a staff
----	-----	--	----------------------

- Admin ready to assign a staff member.

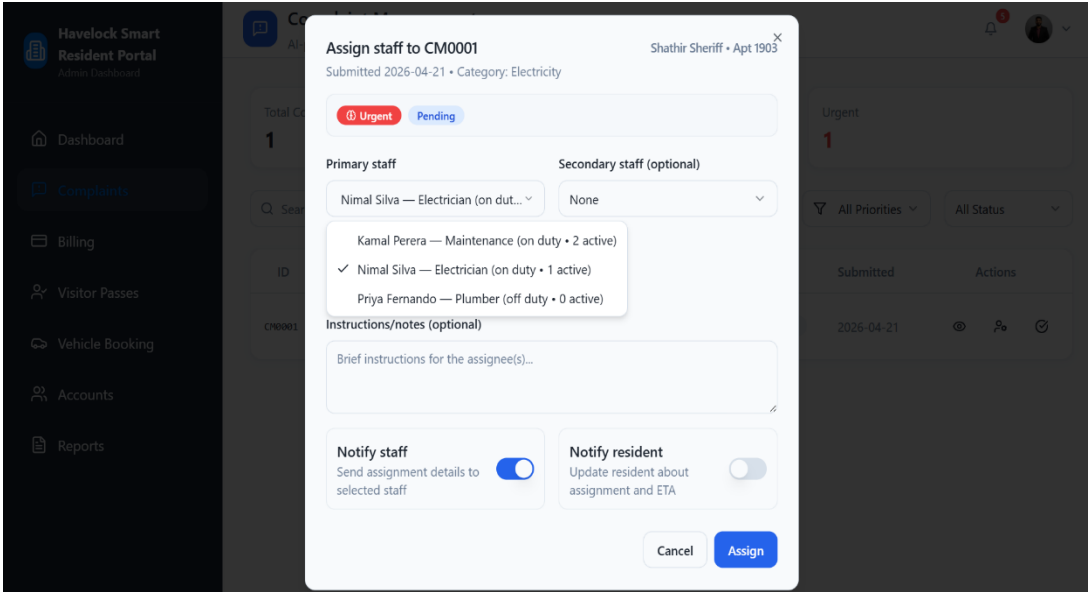


Figure 36. Evidence 15

- Admin assigned a staff and get a message.

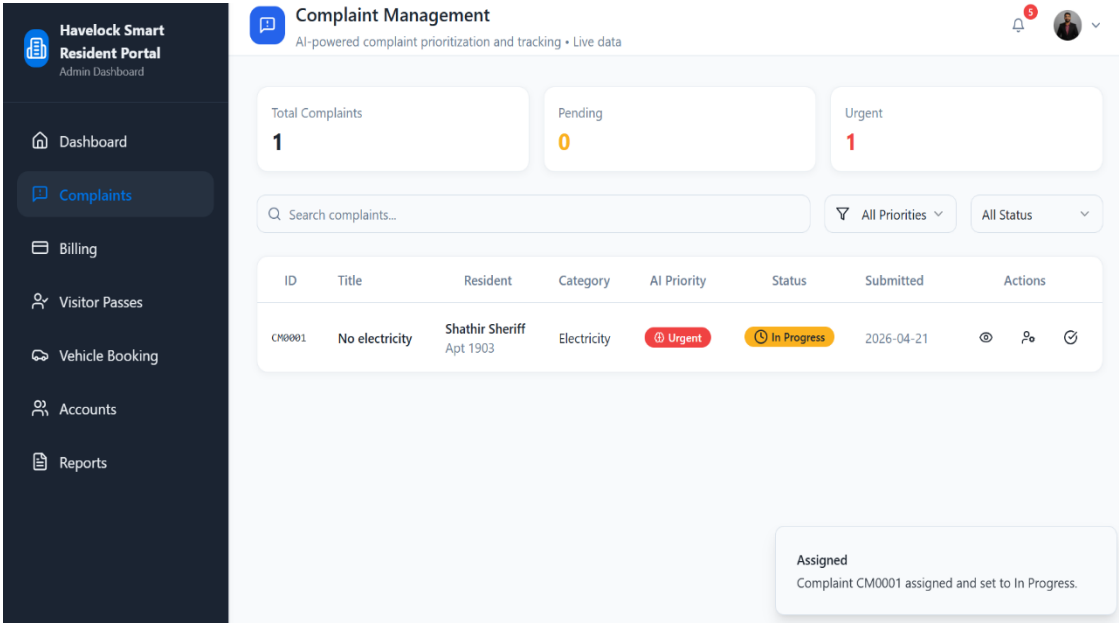


Figure 37. Evidence 16

6.3.3 Visitor Management

Table 14. Test Case 06

Test Case: Visitor Pass generation Pre-condition: Resident in Visitor Passes page			
TC#	Sequence#	Description	Expected results
13	6.1	Submit without completing the form.	Message appear: Please fill in this field.

- A message appears and didn't submit the form.

Havelock Smart Resident Portal
Residents Dashboard

- Dashboard
- Complaints
- Visitor Passes
- Parcels
- Vehicle Booking
- Billing

Generate Visitor Pass

Tower Name: Peterson Tower Apartment Number: 1903

Visitor Name: e.g., John Smith Phone: 0714876345

Please fill in this field.

Purpose: e.g., Family Visit, Delivery

Arrival Date: 04/20/2026 Arrival Time: 06:12 PM

Pass will be valid for 24 hours from the selected arrival date and time.

Vehicle Number (optional): e.g., ABC-1234

+ Generate Pass **Reset**

Active Pass
No active visitor pass. Generate a new pass to see details here.

Figure 40. Evidence 19

14	6.2	Successfully generated a visitor pass.	6-digit visitor pass created.
----	-----	--	-------------------------------

- Receive a 6-digit 24-hour visitor pass.

Havelock Smart Resident Portal
Resident Dashboard

- Dashboard
- Complaints
- Visitor Passes
- Parcels
- Vehicle Booking
- Billing

Generate Visitor Pass

Tower Name: Peterson Tower Apartment Number: 1903

Visitor Name: John smith Phone: 0714876345

Purpose: visiting

Arrival Date: 04/20/2026 Arrival Time: 06:12 PM

Pass will be valid for 24 hours from the selected arrival date and time.

Vehicle Number (optional): e.g., ABC-1234

+ Generate Pass **Reset**

Active Pass
526507 **Copy Code**

Visitor: John smith Phone: 0714876345

Purpose: visiting Tower: Peterson Tower

Apartment: 1903 Vehicle: —

Arrival: 20/04/2026 18:12 Valid Until: 21/04/2026 18:12

Expire Pass

✓ Visitor pass generated for 24 hours.

Figure 41. Evidence 20

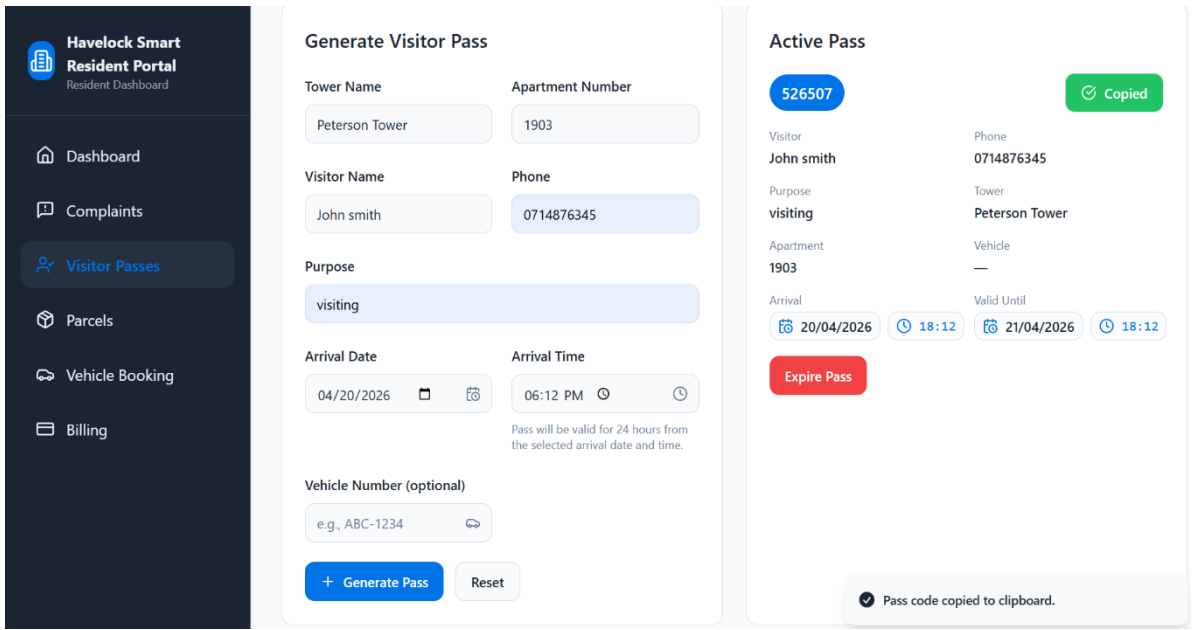
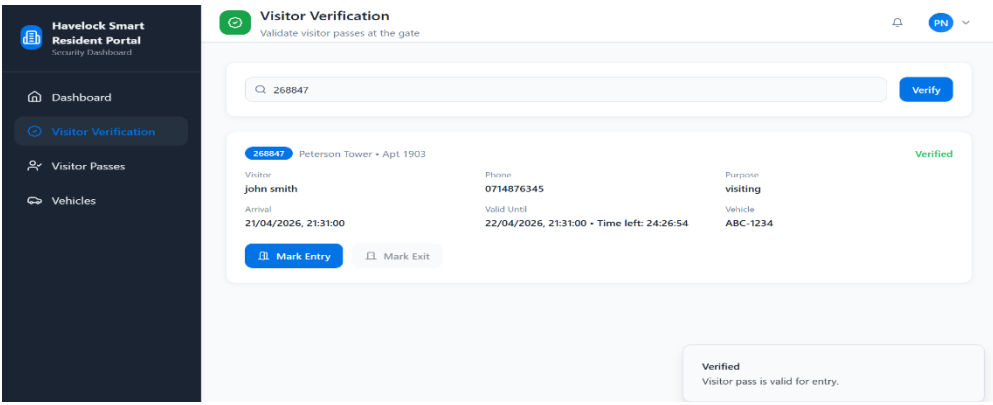
15	6.3	Click the Copy Code button.	Pass code copied.
Successfully copied the passcode.  The screenshot shows the 'Havelock Smart Resident Portal' interface. On the left is a dark sidebar with navigation links: Dashboard, Complaints, Visitor Passes (selected), Parcels, Vehicle Booking, and Billing. The main content area is divided into two panels. The left panel, titled 'Generate Visitor Pass', contains form fields for Tower Name (Peterson Tower), Apartment Number (1903), Visitor Name (John smith), Phone (0714876345), Purpose (visiting), Arrival Date (04/20/2026), and Arrival Time (06:12 PM). There is a 'Vehicle Number (optional)' field and a '+ Generate Pass' button. The right panel, titled 'Active Pass', displays the generated pass code '526507' in a blue circle, a green 'Copied' button, and visitor details: John smith, 0714876345, visiting, Peterson Tower, 1903. It also shows arrival and valid until times. A red 'Expire Pass' button is at the bottom. A toast message at the bottom right states 'Pass code copied to clipboard.'			
Figure 42. Evidence 21			

Table 15. Test Case 07

Test Case: Visitor Verification Pre-condition: Security in Visitor Verification page			
TC#	Sequence#	Description	Expected results
16	7.1	Enter 6-digit number correctly and click verify button.	Verified.
Display the information of the visitor  The screenshot shows the 'Havelock Smart Resident Portal' interface. The sidebar is the same as in Figure 42, with 'Visitor Verification' selected. The main content area is titled 'Visitor Verification' with the subtitle 'Validate visitor passes at the gate'. It features a search bar with the number '268847' and a 'Verify' button. Below the search bar, a card displays the verified visitor information: Peterson Tower - Apt 1903, Visitor John smith, Phone 0714876345, Purpose visiting, Arrival 21/04/2026, 21:31:00, Valid Until 22/04/2026, 21:31:00, and Vehicle ABC-1234. There are 'Mark Entry' and 'Mark Exit' buttons. A green 'Verified' status is shown in the top right corner of the card. A toast message at the bottom right states 'Verified Visitor pass is valid for entry.'			
Figure 43. Evidence 22			

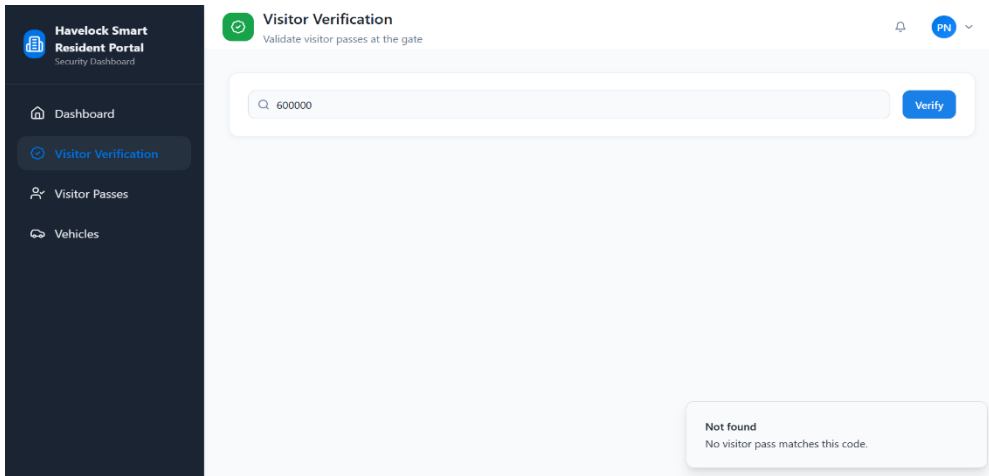
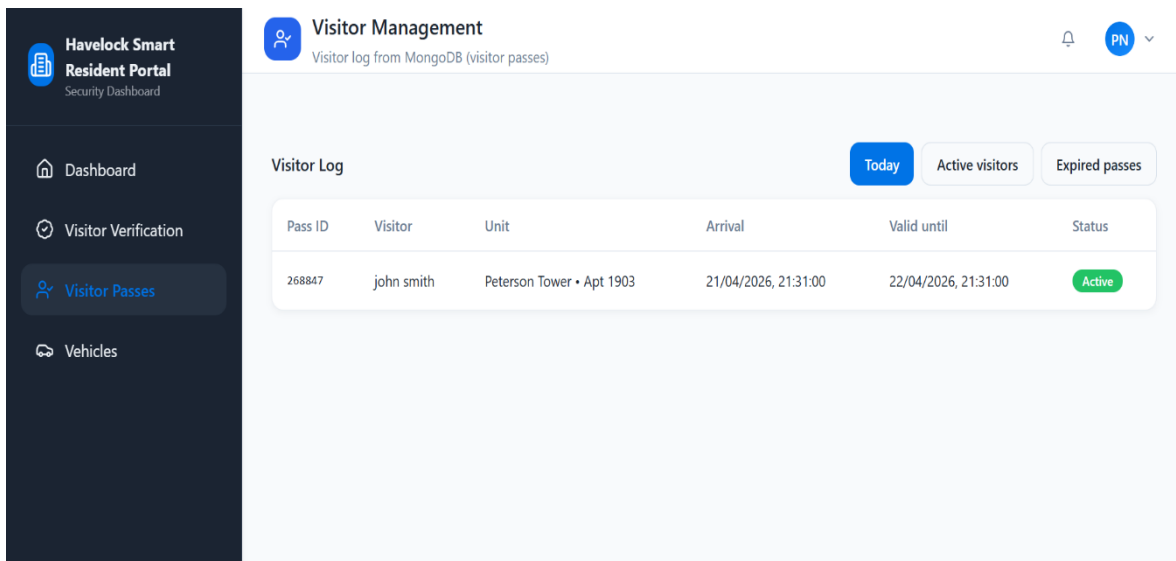
17	7.2	Enter an invalid number.	Not found.
No result found and receive a message.  Figure 44. Evidence 23			
18	7.3	Security view Active passes.	Can see the active passes.
Security can see the active passes in visitor log in visitor passes page.  Figure 45. Evidence 24			
19	7.4	Security view Expired passes.	Can see the expired passes.
Security can see the expired passes in visitor log in visitor passes page.  Figure 46. Evidence 25			

Table 16. Test Case 08

Test Case: Vehicle Verification
Pre-condition: Security in Vehicle page

TC#	Sequence#	Description	Expected results
20	8.1	Enter the correct vehicle number.	Vehicle number verified.

- Vehicle detail displayed.

The screenshot shows the 'Havelock Smart Resident Portal' with a sidebar menu containing 'Dashboard', 'Visitor Verification', 'Visitor Passes', and 'Vehicles'. The main content area is titled 'Vehicle Management' and 'Verification and logs'. It features a 'Vehicle Verification' section with a text input field containing 'ABC-1234' and a 'Verify' button. Below this, there are fields for 'Vehicle number' (ABC-1234), 'Resident / visitor vehicle tag' (visitor), and 'Parking slot (if applicable)' (e.g., B2-17). There are also 'Mark entry' and 'Mark exit' buttons. A 'Vehicle Log' table is displayed below, showing a single entry for vehicle ABC-1234 with tag 'visitor' and status 'Verified'. A toast message at the bottom right states 'Vehicle found Tagged as visitor.'

Figure 47. Evidence 26

21	8.2	Mark Entry.	Entry recorded.
----	-----	-------------	-----------------

- Entry recorded and can see the status as inside in visitor log.

The screenshot shows the same 'Havelock Smart Resident Portal' interface. In the 'Vehicle Verification' section, the 'Verify' button has been clicked, and the 'Mark entry' button is now active. The 'Vehicle Log' table now shows the entry for vehicle ABC-1234 with tag 'visitor', entry time '21/04/2026, 22:03:00', and status 'Inside'. A toast message at the bottom right states 'Entry recorded Entry saved for ABC-1234.'

Figure 48. Evidence 27

22	8.3	Mark exit.	Exit recorded.
----	-----	------------	----------------

- Exit recorded and exit time can be seen in visitor log.

The screenshot shows the 'Vehicle Management' section of the 'Havelock Smart Resident Portal'. The left sidebar contains navigation links: Dashboard, Visitor Verification, Visitor Passes, and Vehicles (highlighted). The main content area is titled 'Vehicle Management' with the subtitle 'Verification and logs'. It features a 'Vehicle Verification' form with a text input for 'ABC-1234' and a 'Verify' button. Below the form, the 'Vehicle number' is 'ABC-1234', the 'Resident / visitor vehicle tag' is 'visitor', and the 'Parking slot (if applicable)' is 'B2-17'. There are 'Mark entry' and 'Mark exit' buttons. Below this is a 'Vehicle Log' table with columns: Vehicle, Tag, Entry time, Exit time, Parking slot, and Status. The table contains one entry for vehicle 'ABC-1234' with tag 'visitor', entry time '21/04/2026, 22:03:00', exit time '21/04/2026, 22:06:57', parking slot 'B2-17', and status 'Verified'. A notification box at the bottom right states 'Exit recorded' and 'Exit saved for ABC-1234.'

Figure 49. Evidence 28

23	8.4	Enter with invalid vehicle number.	Unauthorized.
----	-----	------------------------------------	---------------

- Unauthorized.

The screenshot shows the 'Vehicle Management' section of the 'Havelock Smart Resident Portal'. The left sidebar contains navigation links: Dashboard, Visitor Verification, Visitor Passes, and Vehicles (highlighted). The main content area is titled 'Vehicle Management' with the subtitle 'Verification and logs'. It features a 'Vehicle Verification' form with a text input for 'BMW-10000' and a 'Verify' button. Below the form, the 'Vehicle number' is 'BMW-10000', the 'Resident / visitor vehicle tag' is 'unauthorized' with a red error icon and the text 'Unauthorized', and the 'Parking slot (if applicable)' is 'e.g., B2-17'. There are 'Mark entry' and 'Mark exit' buttons.

Figure 50. Evidence 29

Table 17. Test Case 09

Test Case: Manual-Entry (Walk-in) Pre-condition: Reception in Visitor Passes page			
TC#	Sequence#	Description	Expected results
24	9.1	Submit with incomplete details.	A message appears which telling to fill all the details.

- Manual Entry fails.

The screenshot shows the 'Visitor Passes' section of the 'Havelock Smart Resident Portal'. The 'Manual Entry (Walk-in)' form has the following fields: Visitor Name (shathir), NIC / ID (200132401878), Apartment (e.g., 2526), and Entry time (optional) (04/21/2026 10:52 PM). Below the form is a blue 'Add Entry' button. The 'Visitor Log' table below shows three entries for 'shathir' with status 'Expired'. A tooltip message says 'Please provide name, NIC/ID, and apartment.'

Visitor Name	NIC / ID	Apartment	Entry time (optional)
shathir	200132401878	e.g., 2526	04/21/2026 10:52 PM

Visitor Log

Visitor	NIC / ID	Apartment	Entry	Exit	Status	Actions
shifa	200732401878	Apt 1903	18/04/2026, 14:47:00	21/04/2026, 22:10:34	Expired	
shathir	200132401878	Apt 1903	26/03/2026, 14:50:00	21/04/2026, 22:10:33	Expired	
shathir	200132401878	Apt 1903	26/03/2026, 08:50:00	26/0		

Figure 51. Evidence 30

25	9.2	Submit with complete details.	Manual entry was added.
----	-----	-------------------------------	-------------------------

- Added successfully and message appears.

The screenshot shows the 'Manual Entry (Walk-in)' form with complete details: Visitor Name (e.g., Jane Doe), NIC / ID (e.g., 1995123...), Apartment (e.g., 2526), and Entry time (optional) (mm/dd/yyyy --:--:--). Below the form is a blue 'Add Entry' button. The 'Visitor Log' table below shows three entries for 'shathir' with status 'Active'. A tooltip message says 'Manual visitor entry added.'

Visitor Name	NIC / ID	Apartment	Entry time (optional)
e.g., Jane Doe	e.g., 1995123...	e.g., 2526	mm/dd/yyyy --:--:--

Visitor Log

Visitor	NIC / ID	Apartment	Entry	Exit	Status	Actions
shathir	200132401878	Apt 1903	22/04/2026, 03:53:00	—	Active	Mark Exit
Hasitha Perera	—	Apt 1903	22/04/2026, 01:08:32	—	Expired	Mark Exit
Shathir Sheriff	—	Aot 1903	22/04/2026, 01:03:39	—	Expired	Mark Exit

Figure 52. Evidence 31

26	9.3	Mark exit.	Mark exited.
Exit marked. Figure 53. Evidence 32			

6.3.4 Inter-Vehicle Booking Management

Table 18. Test Case 10

Test Case: Booking an Inter-vehicle Pre-condition: Resident in Vehicle Booking page			
TC#	Sequence#	Description	Expected results
27	10.1	Submit the form without completing the require details.	A Message appears.
Resident still in the request booking form. Figure 54. Evidence 33			

28	10.2	Request booking with sharing all the details.	Successfully requested a vehicle booking.
----	------	---	---

- Receive a success message for vehicle booking.

The screenshot shows the 'Havelock Smart Resident Portal' with a sidebar menu containing Dashboard, Complaints, Visitor Passes, Parcels, Vehicle Booking (highlighted), and Billing. The main content area is titled 'Create and manage your bookings' and features a 'New Booking' form. The form includes fields for Vehicle Number (e.g., ABC-1234), Date (04/20/2026), Time (07:29 PM), From (e.g., Tower A), To (e.g., Airport), and Purpose (e.g., Airport drop). Below the form are 'Request Booking' and 'Reset' buttons. A success message 'Vehicle booking requested.' is displayed at the bottom right. Below the form, there is a section for 'Upcoming Bookings' with a count of 1.

Figure 55. Evidence 34

- Can view in the upcoming booking section in the vehicle booking page.

The screenshot shows the 'Havelock Smart Resident Portal' with the same sidebar menu. The main content area displays the 'Upcoming Bookings' section, which includes a table with the following data:

Booking	Vehicle	When	Route	Purpose	Status	Actions
69e628c83dad673136dffd5f	ABC-1234	2026-04-20 19:29	Peterson → Main gate	visiting	Pending	Cancel

The table has a count of 1 booking. The 'Actions' column contains a red 'Cancel' button.

Figure 56. Evidence 35

6.3.5 Parcels Management

Table 19. Test Case 11

Test Case: Send parcel notification to Resident

Pre-condition: Reception in Parcels page

TC#	Sequence#	Description	Expected results
29	11.1	Sending notification with a parcel id which already exists.	A parcel with this ID already exists for this tower.

- Display a message.

Figure 57. Evidence 36

30	11.2	Sending notification to resident for parcel received.	Passed the message successfully.
----	------	---	----------------------------------

- Display a success message to reception.

Figure 58. Evidence 37

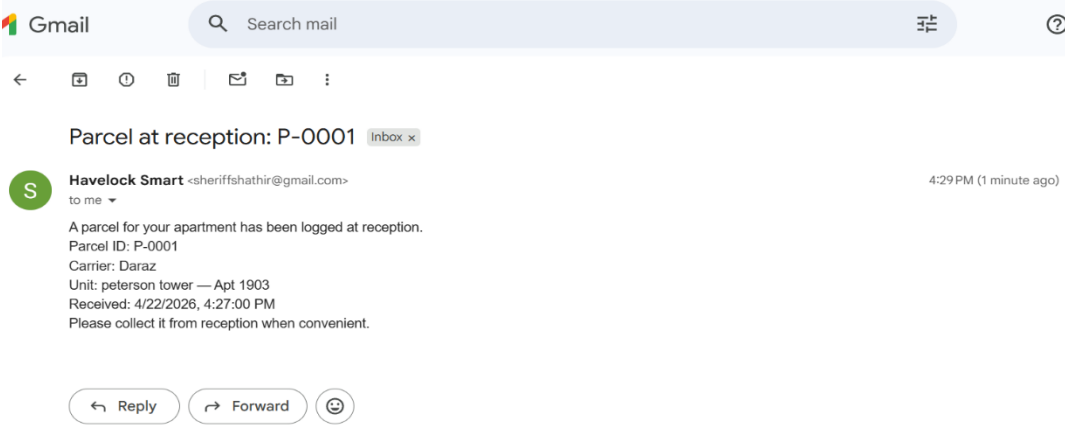
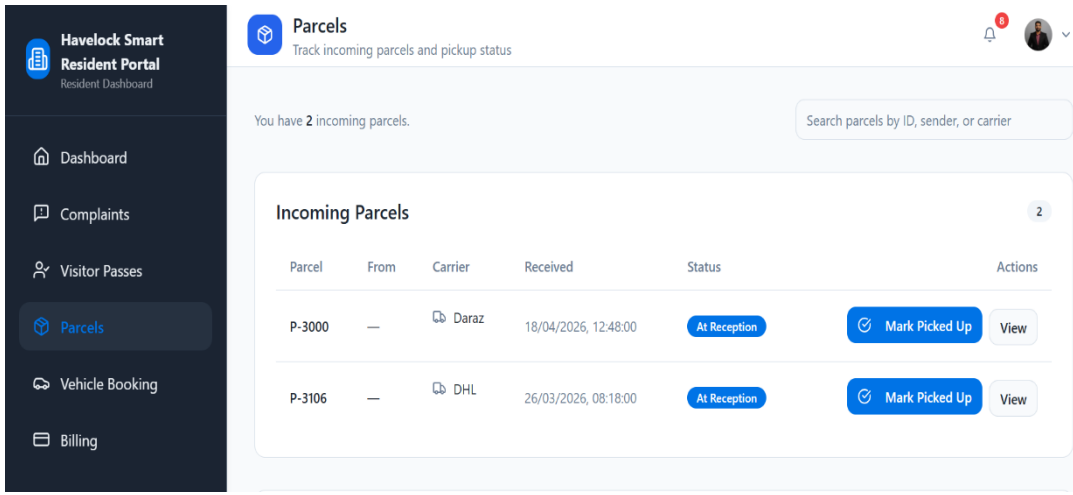
31	11.3	Sent an email notification to relevant resident.	Email notification sends successfully.
Resident receive an email notification.  Figure 59. Evidence 38			

Table 20. Text Case 12

Test Case: Parcels Tracking Pre-condition: Resident in Parcels page			
TC#	Sequence#	Description	Expected results
32	12.1	View Incoming Parcels.	Able to see if there is a parcel received at reception.
Can see incoming parcels in parcels page.  Figure 60. Evidence 39			

- Can view the details of the parcel.

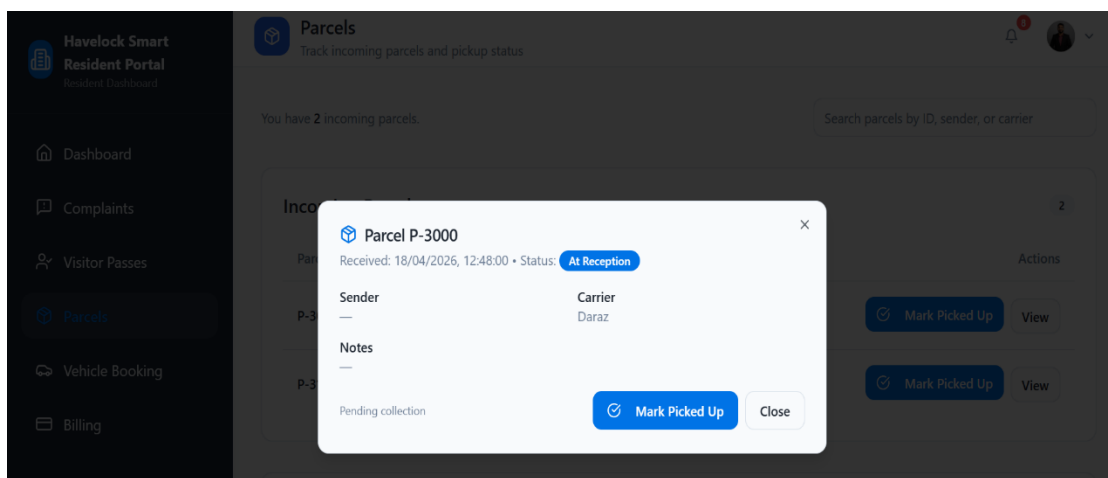


Figure 61. Evidence 40

33	12.2	Marked Picked Up.	Marked as collected.
----	------	-------------------	----------------------

- Received a message as collected the parcel.

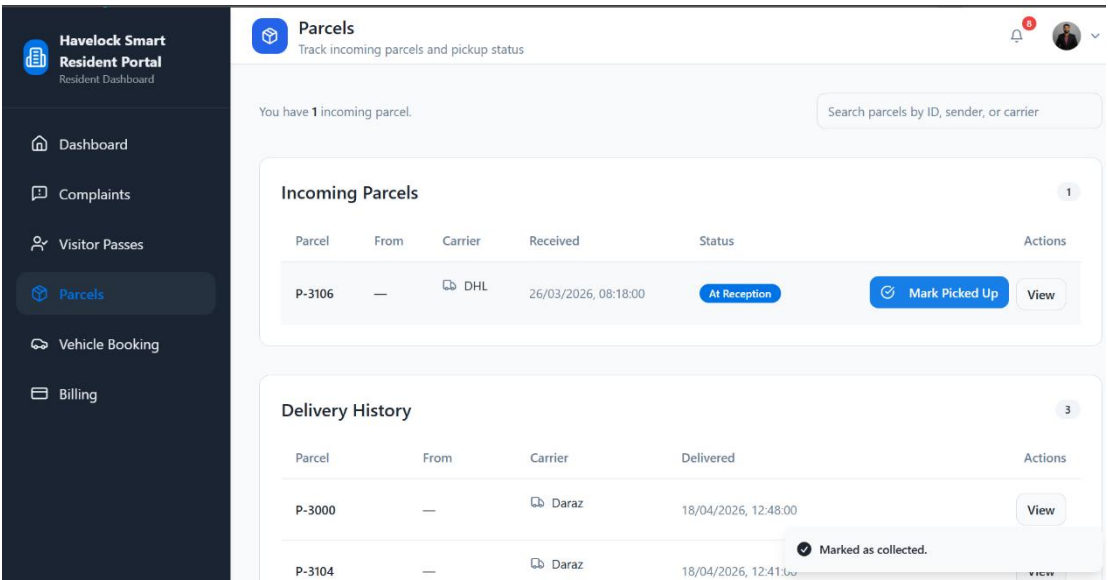
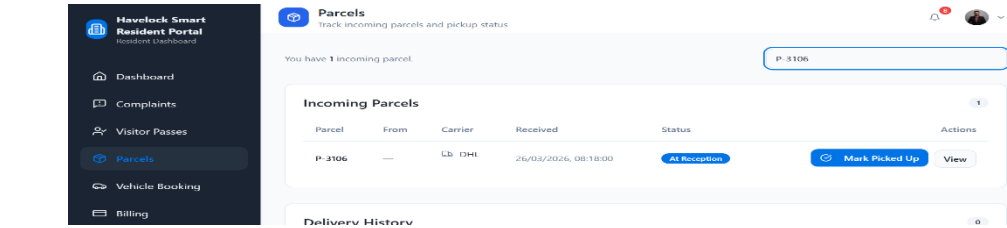
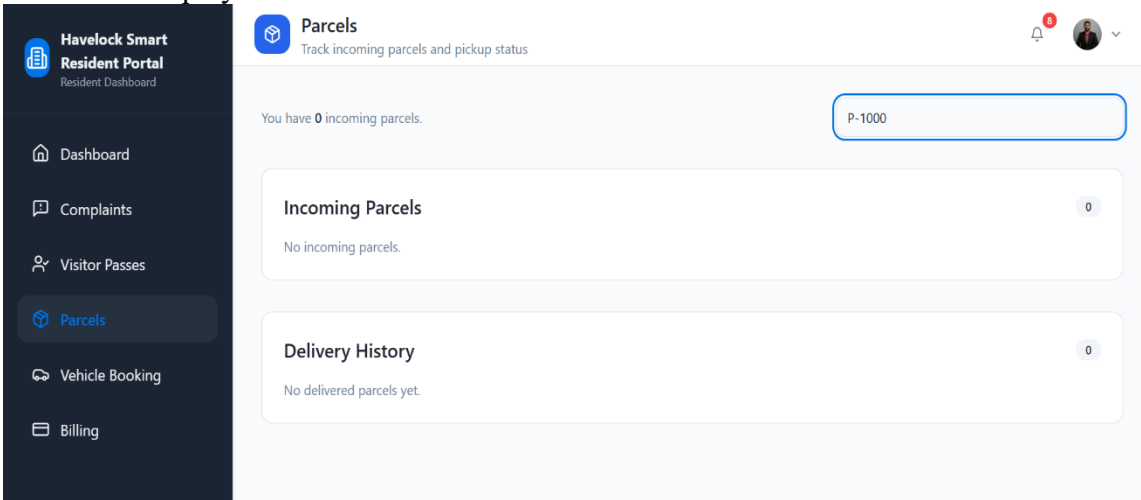


Figure 62. Evidence 41

34	12.3	Search by Parcel ID.	The searched parcel displayed.
Display the parcel which searched.  The screenshot shows the 'Havelock Smart Resident Portal' with a sidebar menu containing Dashboard, Complaints, Visitor Passes, Parcels (selected), Vehicle Booking, and Billing. The main content area is titled 'Parcels' and 'Track incoming parcels and pickup status'. It displays 'You have 1 incoming parcel.' and a search bar with 'P-3106'. Below is a table for 'Incoming Parcels' with one entry: Parcel P-3106, From —, Carrier LB DHL, Received 26/03/2026, 08:18:00, Status At Reception, and Actions Mark Picked Up and View. A 'Delivery History' section below shows 0 items. Figure 63. Evidence 42			
35	12.4	Search by Parcel ID which doesn't exist.	No parcels displayed.
No results display.  The screenshot shows the same 'Havelock Smart Resident Portal' interface. The search bar now contains 'P-1000'. The 'Incoming Parcels' section displays '0' and 'No incoming parcels.' The 'Delivery History' section also displays '0' and 'No delivered parcels yet.' Figure 64. Evidence 43			

6.3.6 Billing Management

Table 21. Test Case 13

Test Case: Billing Management - Admin

Pre-condition: Admin in Billing page

TC#	Sequence#	Description	Expected results
36	13.1	Create a bill.	Bill created.

- Fill the create new bill form.

Figure 65. Evidence 44

- Bill created.

Bill ID	Apartment	Resident	Amount	Due Date	Status	Actions
B0013	1903	Shathir Sheriff	LKR 50,000	2026-04-23	Unpaid	[Icon] [Icon] [Icon] [Icon]
B0012	1903	Shathir Sheriff	LKR 15,000	2026-04-30	Paid	[Icon] [Icon]
B0011	1903	Shathir Sheriff	LKR 15,000	2026-04-30	Unpaid	[Icon] [Icon] [Icon] [Icon]
B0010	1903	Shathir Sheriff	LKR 15,000	2026-02-28	Unpaid	[Icon] [Icon] [Icon] [Icon]
B0009	1903	Shathir Sheriff	LKR 15,000	2026-02-28	Unpaid	[Icon] [Icon] [Icon] [Icon]

Figure 66. Evidence 45

37	13.2	Download PDF document of billing summary.	Billing summary pdf download.
----	------	---	-------------------------------

- PDF document downloaded.

Havelock Smart Resident Portal
Admin Dashboard

Billing & Payment Management
Track and manage monthly billing

billing-report (11).pdf
14.1 KB • Done

Total Bills **LKR 125,000** Collected **LKR 15,000** Pending **LKR 110,000** Overdue **0**

Search by resident or apartment... All Status Export Report Refresh Generate Bills

Bill ID	Apartment	Resident	Amount	Due Date	Status	Actions
B0013	1903	Shathir Sheriff	LKR 50,000	2026-04-23	Unpaid	[Icon] [Icon] [Icon] [Icon]
B0012	1903	Shathir Sheriff	LKR 15,000	2026-04-30	Paid	[Icon] [Icon]
B0011	1903	Shathir Sheriff	LKR 15,000	2026-04-30	Unpaid	[Icon] [Icon] [Icon] [Icon]
B0010	1903	Shathir Sheriff	LKR 15,000	2026-02-28	Unpaid	[Icon] [Icon] [Icon] [Icon]

Figure 67. Evidence 46

Billing Report
Generated at 23/04/2026, 12:29:55

Total Bills: LKR 125,000.00 Collected: LKR 15,000.00 Pending: LKR 110,000.00 Overdue: 0

Bill ID	Apartment	Resident	Amount	Due Date	Status	Month
69e9bedf687201bcbf4db1e8	1903	Shathir Sheriff	LKR 50,000.00	2026-04-23	Unpaid	April 2026
69e331e99f1e31dc255fd69e	1903	Shathir Sheriff	LKR 15,000.00	2026-04-30	Paid	April 2026
69e330dd5134d406d72881c8	1903	Shathir Sheriff	LKR 15,000.00	2026-04-30	Unpaid	April 2026
69a12493174acb4cca60023	1903	Shathir Sheriff	LKR 15,000.00	2026-02-28	Unpaid	February 2026
699d2de50b3924cb495b441a	1903	Shathir Sheriff	LKR 15,000.00	2026-02-28	Unpaid	February 2026
699d2d800b3924cb495b440f	1903	Shathir Sheriff	LKR 15,000.00	2026-02-27	Unpaid	February 2026

Figure 68. Evidence 47

Table 22. Test Case 14

Test Case: Billing Management - Resident
Pre-condition: Resident in Billing page

TC#	Sequence#	Description	Expected results
38	13.1	Can make as paid the bill and download the pdf file of the bill summary.	Payment recorded and can download the bill summary.

- Evidences.

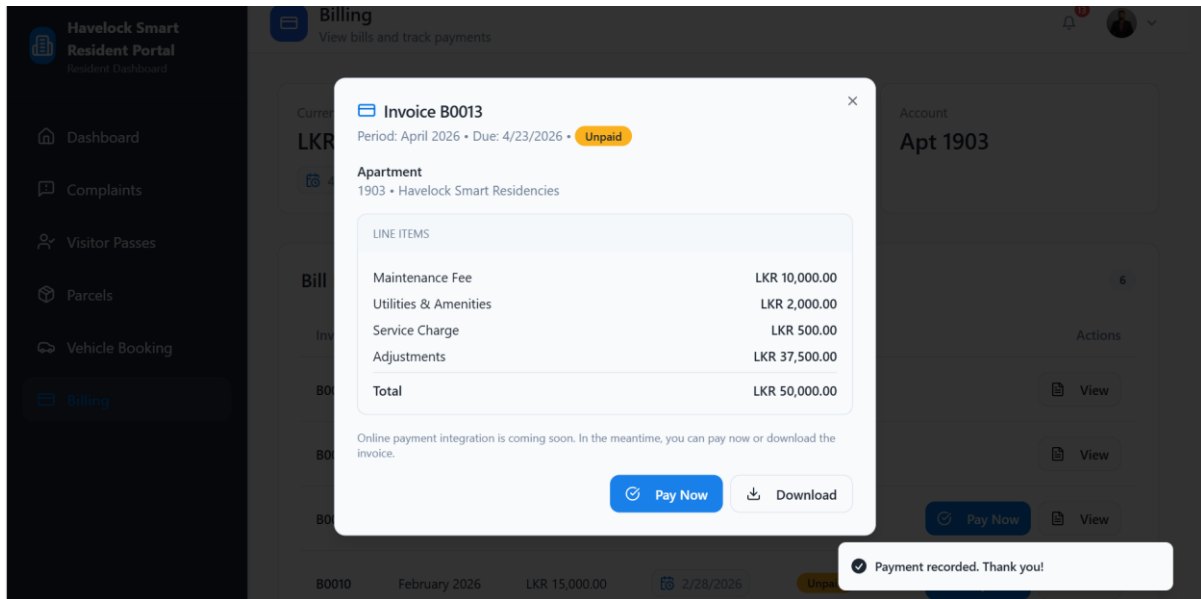


Figure 69. Evidence 48

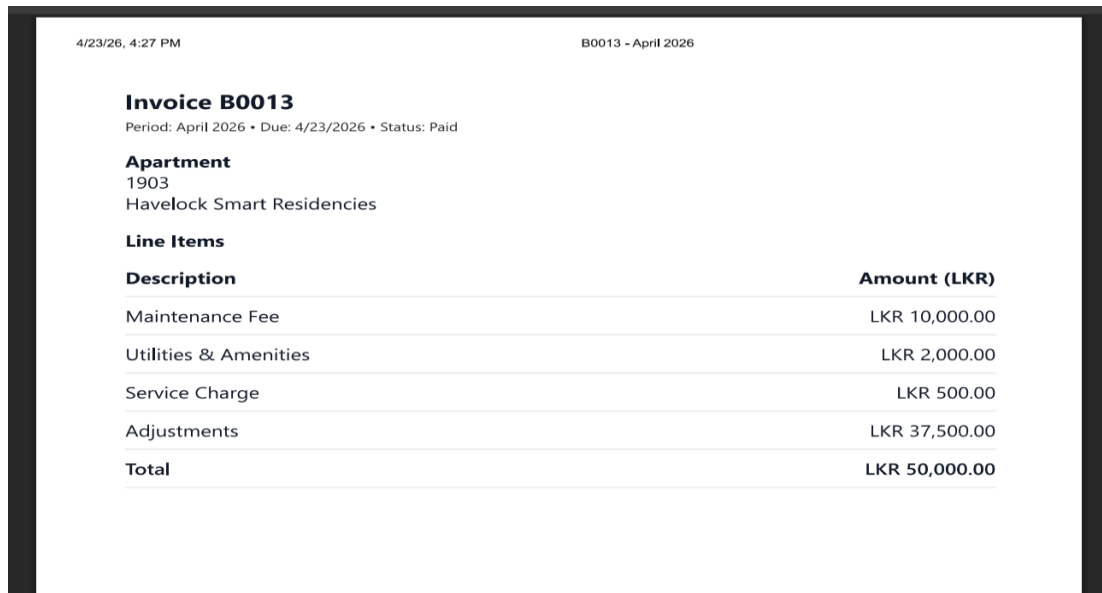


Figure 70. Evidence 49

6.3.7 Accounts Management

Table 23. Test Case 15

Test Case: Reset Password Pre-condition: User in Settings page			
TC#	Sequence#	Description	Expected results
39	15.1	Update password including invalid current password.	Current password is incorrect.

- Evidence.

Security
Change your password

Current Password
.....

New Password
.....

Confirm New Password
.....

Update Password

Tip: Use at least 8 characters with a mix of letters, numbers, and symbols.

Notification Preferences
Configure how you receive notifications

Email Notifications
Receive email alerts for urgent comp

Push Notifications
Browser push notifications for new a

Update failed
Current password is incorrect.

Figure 71. Evidence 50

40	15.2	Update password with new password and enter a different password in confirm new password box.	New password and confirmation don't match.
----	------	---	--

- Evidence.

Security
Change your password

Current Password
.....

New Password
.....

Confirm New Password
.....

Update Password

Tip: Use at least 8 characters with a mix of letters, numbers, and symbols.

Notification Preferences
Configure how you receive notifications

Email Notifications
Receive email alerts for urgent comp

Push Notifications
Browser push notifications for new a

Update failed
New password and confirmation do not match.

Figure 72. Evidence 51

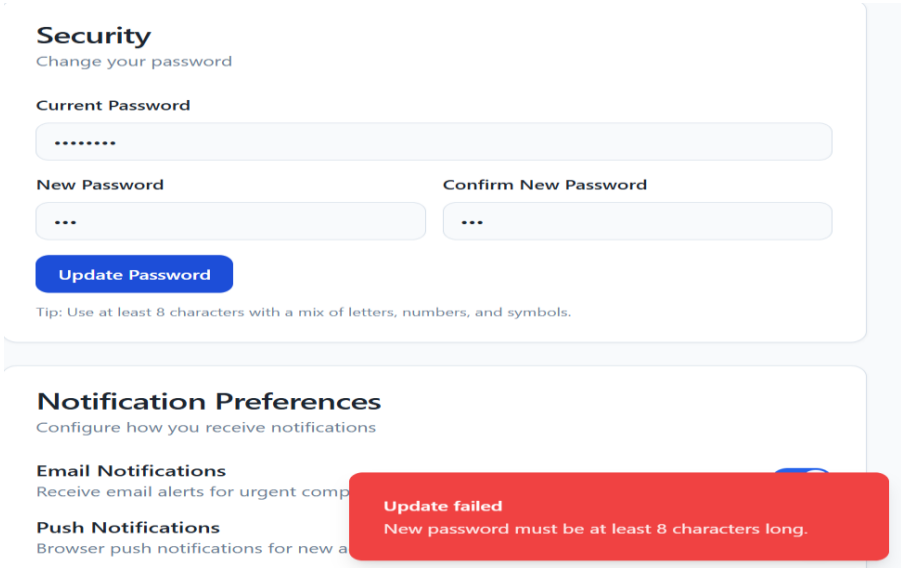
41	15.3	Update the password with new one less than 8 characters.	New password at least 8 characters long.
Evidence.  Security Change your password Current Password New Password ... Confirm New Password ... Update Password Tip: Use at least 8 characters with a mix of letters, numbers, and symbols. Notification Preferences Configure how you receive notifications Email Notifications Receive email alerts for urgent comp Push Notifications Browser push notifications for new a Update failed New password must be at least 8 characters long.			

Figure 73. Evidence 52

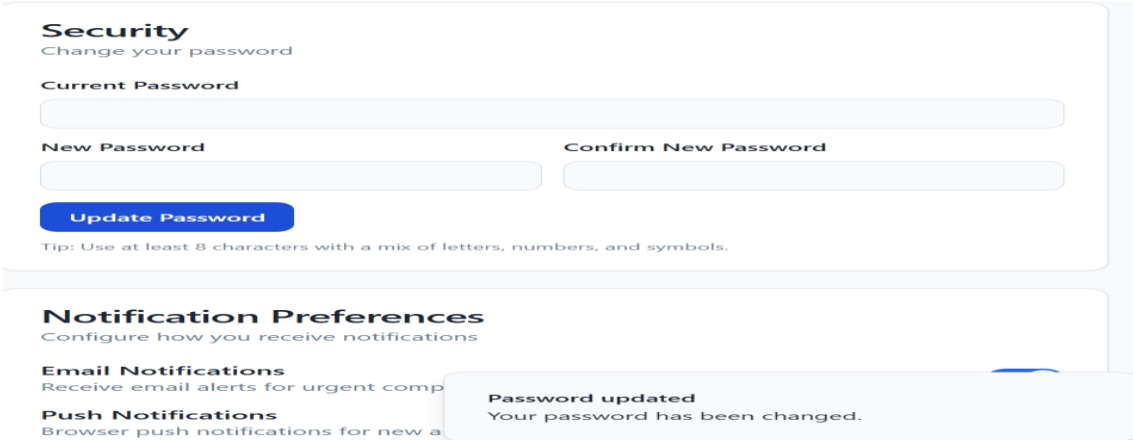
42	15.4	Update the password with correct requirements.	Password updated.
Evidence  Security Change your password Current Password New Password Confirm New Password Update Password Tip: Use at least 8 characters with a mix of letters, numbers, and symbols. Notification Preferences Configure how you receive notifications Email Notifications Receive email alerts for urgent comp Push Notifications Browser push notifications for new a Password updated Your password has been changed.			

Figure 74. Evidence 53

Table 24. Test Case 16

Test Case: Update User profile Pre-condition: User in Settings page			
TC#	Sequence#	Description	Expected results
43	16.1	Save changes of profile in profile settings.	Profile detail updated.

- Evidence.

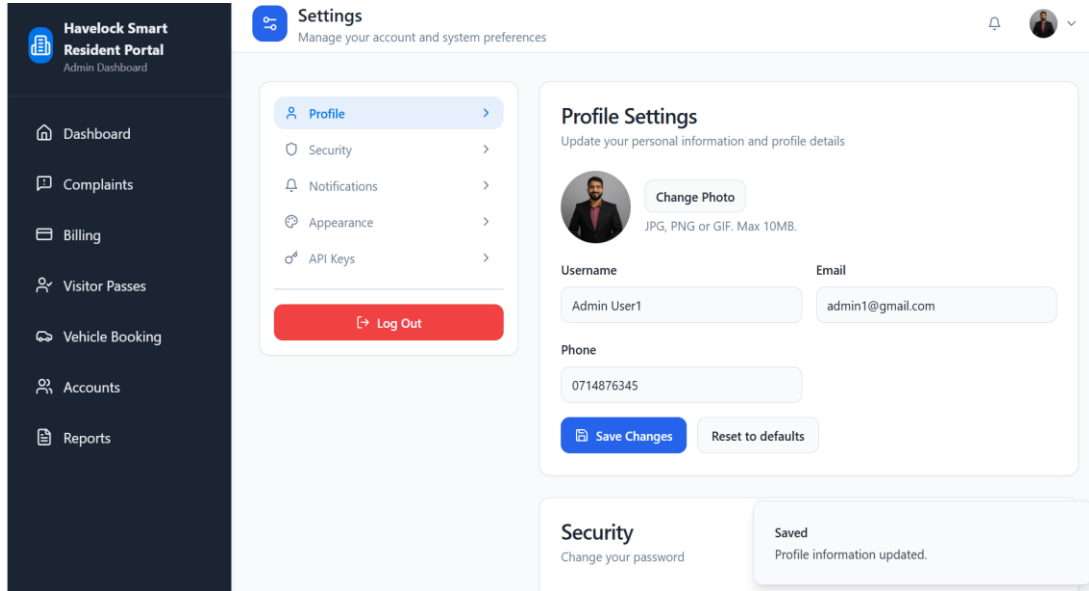


Figure 75. Evidence 54

44	16.2	Reset the profile in profile settings.	Profile reset to defaults.
----	------	--	----------------------------

- Evidence.

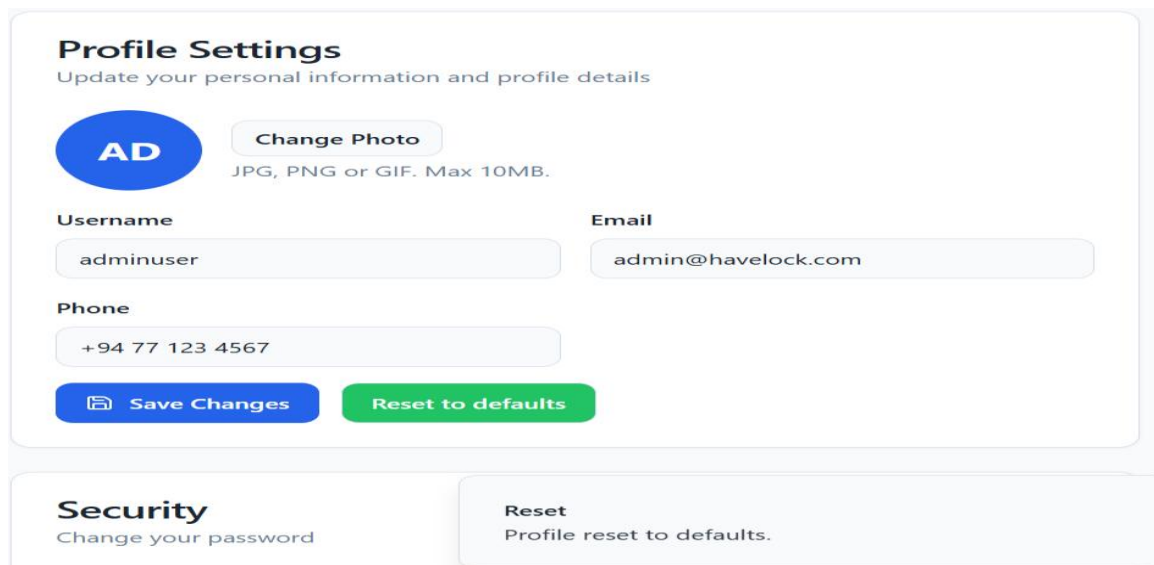


Figure 76. Evidence 55

Table 25. Test Case 17

Test Case: Forget Password Pre-condition: Resident in Login page			
TC#	Sequence#	Description	Expected results
45	17.1	Click forget password and give the Google username.	Reset password link sends to email.

- Evidences.

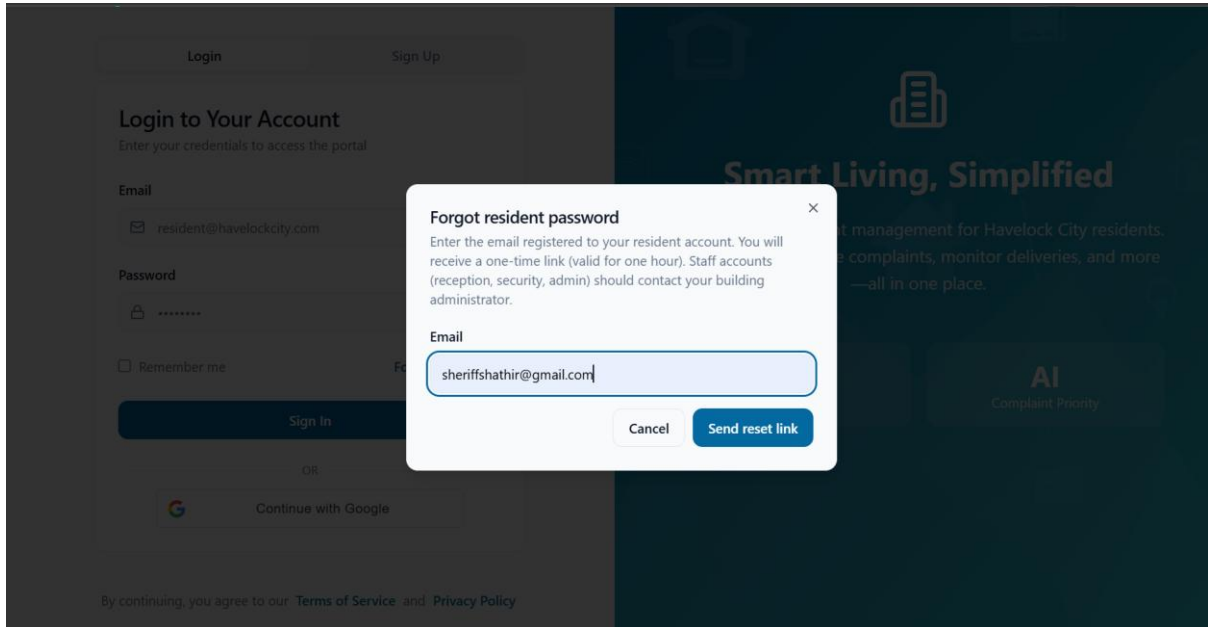


Figure 77. Evidence 56

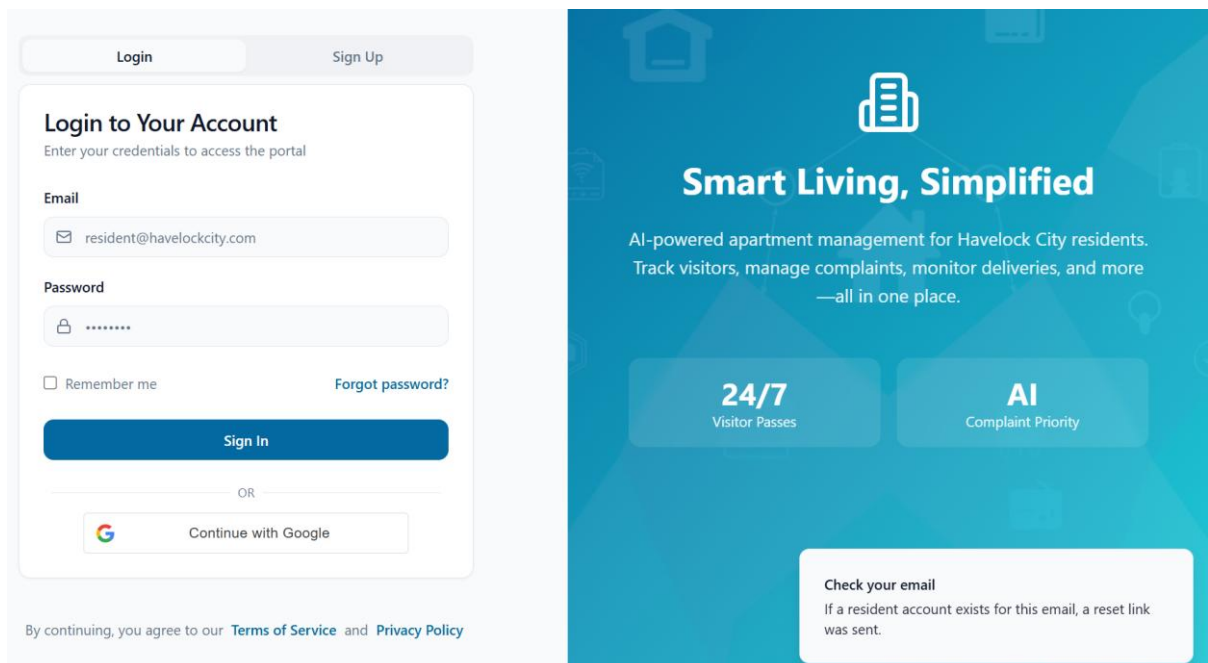


Figure 78. Evidence 57

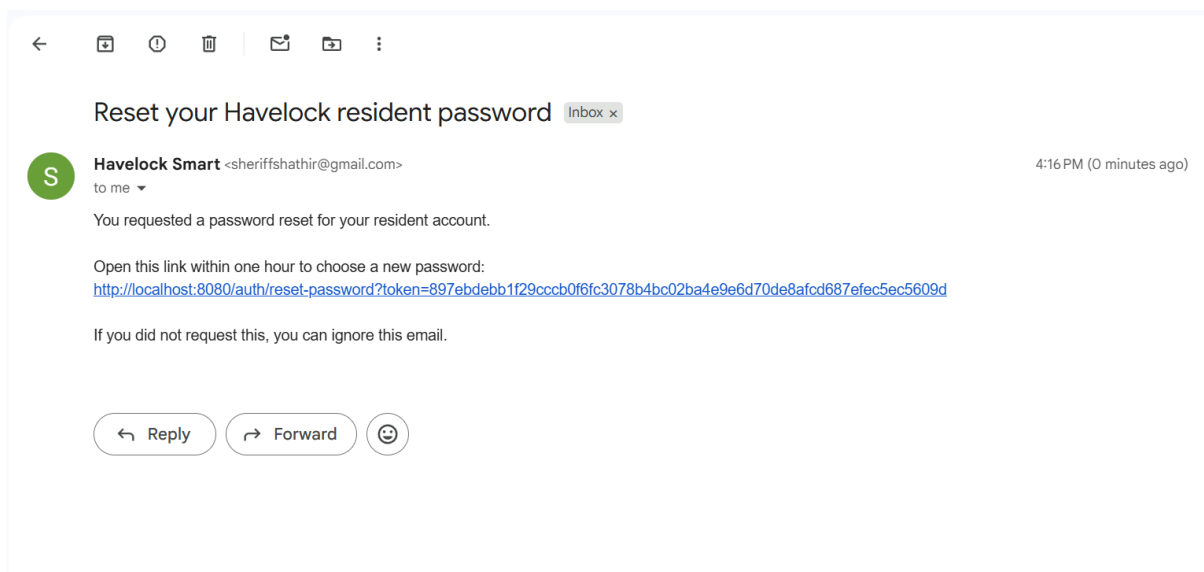


Figure 79. Evidence 58

46	17.2	Submit the reset password form with matching requirements.	Password updated.
----	------	--	-------------------

- Evidence.

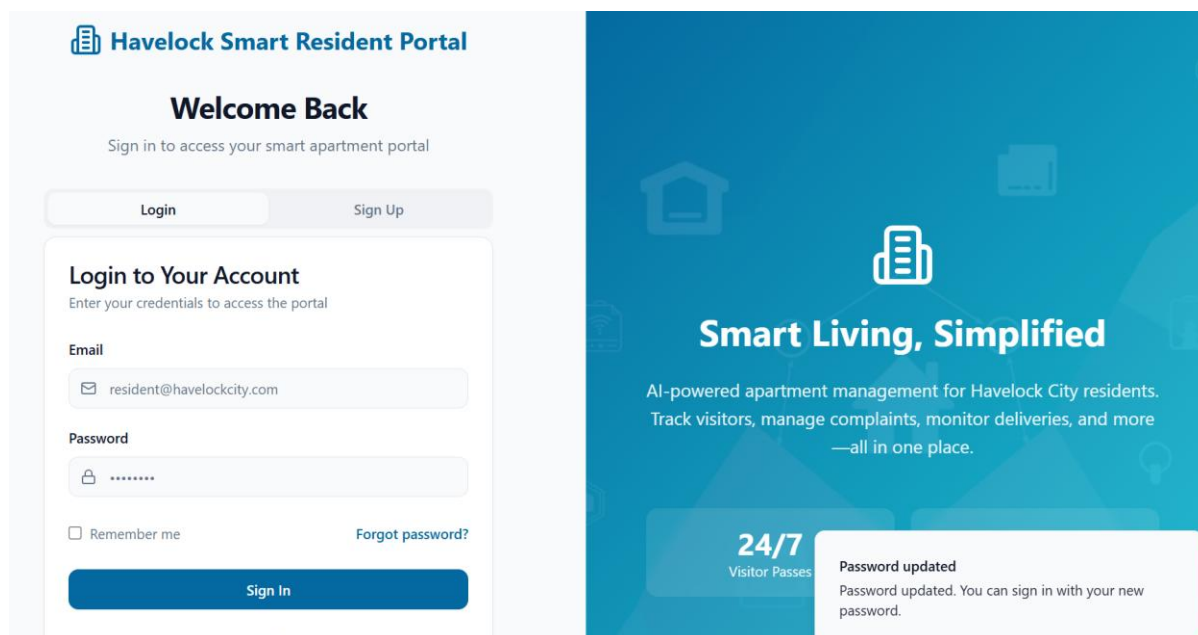


Figure 80. Evidence 59

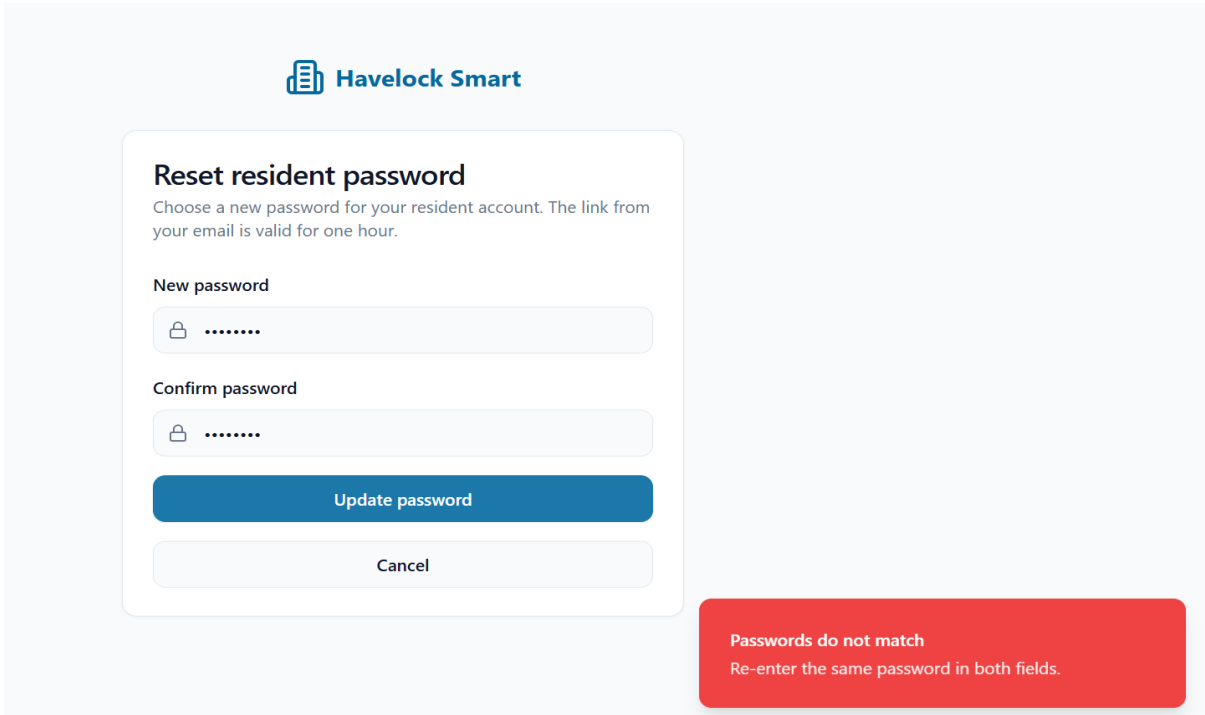
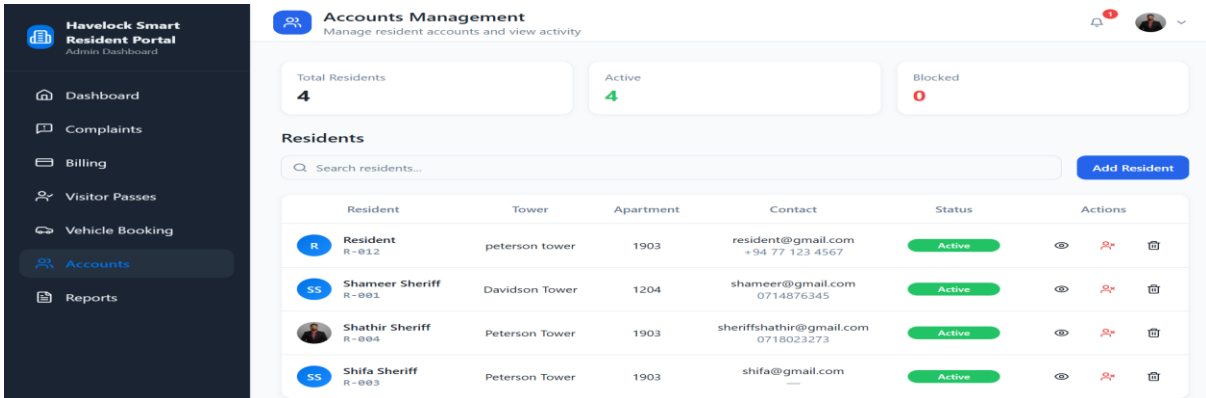
47	17.3	Submit the reset password with invalid details.	Unmatched password.
Evidence. 			
Figure 81. Evidence 60			

Table 26. Test Case 18

Test Case: Admin manage accounts add, view, delete Pre-condition: Admin in accounts page			
TC#	Sequence#	Description	Expected results
48	18.1	Admin view all the user accounts.	View all the accounts.
Evidence 			
Figure 82. Evidence 61			

49	18.2	Admin add a new user account.	User account added.																														
Evidence Total Residents 4 Active 4 Blocked 0 Residents Q Search residents... Add Resident <table><tr><th>Resident</th><th>Tower</th><th>Apartment</th><th>Contact</th><th>Status</th><th>Actions</th></tr><tr><td> R Resident R-012 </td><td>peterson tower</td><td>1903</td><td>resident@gmail.com +94 77 123 4567</td><td>Active</td><td> </td></tr><tr><td> SS Shameer Sheriff R-001 </td><td>Davidson Tower</td><td>1204</td><td>shameer@gmail.com 0714876345</td><td>Active</td><td> </td></tr><tr><td> Shathir Sheriff R-004 </td><td>Peterson Tower</td><td>1903</td><td>sheriffshathir@gmail.com 0718023273</td><td>Active</td><td> </td></tr><tr><td> SS Shifa Sheriff R-003 </td><td>Peterson Tower</td><td>1903</td><td>shifa@gmail.co —</td><td></td><td></td></tr></table> Resident added User Resident added.				Resident	Tower	Apartment	Contact	Status	Actions	R Resident R-012	peterson tower	1903	resident@gmail.com +94 77 123 4567	Active		SS Shameer Sheriff R-001	Davidson Tower	1204	shameer@gmail.com 0714876345	Active		Shathir Sheriff R-004	Peterson Tower	1903	sheriffshathir@gmail.com 0718023273	Active		SS Shifa Sheriff R-003	Peterson Tower	1903	shifa@gmail.co —		
Resident	Tower	Apartment	Contact	Status	Actions																												
R Resident R-012	peterson tower	1903	resident@gmail.com +94 77 123 4567	Active																													
SS Shameer Sheriff R-001	Davidson Tower	1204	shameer@gmail.com 0714876345	Active																													
Shathir Sheriff R-004	Peterson Tower	1903	sheriffshathir@gmail.com 0718023273	Active																													
SS Shifa Sheriff R-003	Peterson Tower	1903	shifa@gmail.co —																														
Figure 83. Evidence 62																																	

50	18.3	Admin delete a user account.	Account deleted.																												
Evidence. Total Residents 3 Active 3 Blocked 0 Residents Q Search residents... Add Resident <table><tr><th>Resident</th><th>Tower</th><th>Apartment</th><th>Contact</th><th>Status</th><th>Actions</th></tr><tr><td> SS Shameer Sheriff R-001 </td><td>Davidson Tower</td><td>1204</td><td>shameer@gmail.com 0714876345</td><td>Active</td><td> </td></tr><tr><td> Shathir Sheriff R-004 </td><td>Peterson Tower</td><td>1903</td><td>sheriffshathir@gmail.com 0718023273</td><td>Active</td><td> </td></tr><tr><td> SS Shifa Sheriff R-003 </td><td>Peterson Tower</td><td>1903</td><td>shifa@gmail.com —</td><td>Active</td><td> </td></tr></table> Admin Q Search admin users... <table><tr><th>Admin</th><th>Contact</th><th>Status</th><th>Actions</th></tr></table> Resident deleted				Resident	Tower	Apartment	Contact	Status	Actions	SS Shameer Sheriff R-001	Davidson Tower	1204	shameer@gmail.com 0714876345	Active		Shathir Sheriff R-004	Peterson Tower	1903	sheriffshathir@gmail.com 0718023273	Active		SS Shifa Sheriff R-003	Peterson Tower	1903	shifa@gmail.com —	Active		Admin	Contact	Status	Actions
Resident	Tower	Apartment	Contact	Status	Actions																										
SS Shameer Sheriff R-001	Davidson Tower	1204	shameer@gmail.com 0714876345	Active																											
Shathir Sheriff R-004	Peterson Tower	1903	sheriffshathir@gmail.com 0718023273	Active																											
SS Shifa Sheriff R-003	Peterson Tower	1903	shifa@gmail.com —	Active																											
Admin	Contact	Status	Actions																												
Figure 84. Evidence 63																															

Chapter 07: Results & Discussion

This chapter discusses the results gained from the design, implementation, and evaluation of the Havelock Smart Resident Portal. The system consists of five functional platforms, namely Havelock (authentication and homepage), Resident, Admin, Reception and Security, in order to automate standard residential functions.

The discussion checks how well the developed platform manages important operational issues, like service handling delays, fragmented stakeholder communication, limited workflow monitoring, and depend on manual results. The research objectives, the system architecture, and the observed workflow behaviour during system use are considered when evaluating the results.

Table 27. Summary of Key Features & Outcomes

Feature	Outcome
Complaint Management	Fully digitalized with AI prioritizing.
Visitor Handling	Real-time tracking enabled.
Parcel Management	Instant notifications to resident.
Billing & Reports	PDF generation and dashboards.

As shown in the Table 27, the outcomes show that Havelock Smart Resident Portal provides a logical digital functional model for apartment community management.

7.1 System Performance & Workflow Efficiency

Ongoing multi-portal tasks with reliable responsiveness and data consistency are provided by the system. Workflows such as resident visitor passes, which verified by security, and parcels, which are apparent to residents, are carried out continuously.

Compared to traditional systems, delays dropped significantly due to API centralized data and notifications.

Table 28. Performance Comparison

Process	Traditional System	Havelock Smart Resident Portal
Complaint Management	Manual, 2-3 days delay.	Automated, less than one hour update.
Visitor Tracking	Paper logs, error-prone.	Real-time digital logs.
Communication	Phone/email delays.	Instant push notification.
Report Generation	Manual Excel.	One-click PDF export.

Table 28 quantifies efficiency gains. The below figure shows a complaint workflow from submission to admin resolution, demonstrating process continuity.

Havelock Smart Resident Portal
Resident Dashboard

- Dashboard
- Complaints
- Visitor Passes
- Parcels
- Vehicle Booking
- Billing

File New Complaint

Fill in the details below to submit your complaint.

Complaint Title *
Water leakage in kitchen

Category *
Maintenance

Description *
Water leakage in kitchen

AI Feedback
AI Priority: Urgent
Based on keywords: leak • category: maintenance
Rules synced: 21:58:15

Submit Complaint Cancel

Havelock Smart Resident Portal
Admin Dashboard

- Dashboard
- Complaints
- Billing
- Visitor Passes

Notifications

Resident-facing alerts (billing, complaints, visitors, and other)

Unread 2 Mark all as read Clear all Refresh

Complaint updates

Complaint received New
We received your complaint "Water leakage in kitchen" (maintenance). Our team will review it shortly.
Resident: Peterson Tower • Apt 1903 • Shathir Sheriff
24/04/2026, 21:59:48 Mark read

Figure 85. Complaint Workflow

7.2 Intelligent Features & Decision Support

A significant advance of the system is the integration of AI-assisted complaint prioritizing, which is carried out through rule-based scoring and keyword analysis. This feature offers immediate feedback on the urgency of the complaint during submission, categorizing issues into specific levels such as Urgent, Medium, Low.

Table 29. AI Prioritization Examples

Input Complaint	Predicted Priority	Reason
Water leakage in the kitchen urgent.	Urgent	Keyword: “Water”, “urgent”
Light bulb not working. (Category: maintenance)	Medium	Keyword: “maintenance”
General inquiry about parking.	Low	No urgency indicators

The AI functionality contributes significantly to operational efficiency by unifying decision-making processes and reducing reliance on manual judgement. Early identification of high-priority issues allows for faster response and improved service quality. Additionally, the systematic prioritization of complaints guarantees consistent processing or related instances and aids administrators in better managing workloads.

But the evaluation also highlights certain limitations. When complaint descriptions are inaccurate or unclear, the existing AI Implementations rely on predefined rules and keyword matching may lower accuracy. Additionally, the system doesn’t automatically get better over time because it lacks adaptive learning capabilities. Unlike such limitations, the AI feature

provides an understandable and open decision-support mechanism, which makes it appropriate for operational settings where understanding and accountability are crucial.

Evidence: core logic (/backend/src/lib/complainPriorityFromRules.js)

```
function computePriorityDetail(rules, category, title, description) {
  const r = normalizeRulesRecord(rules);
  const cat = String(category || "").trim().toLowerCase();
  const text = `${cat} ${String(title || "")} ${String(description || "")}`.toLowerCase();
  let score = 0;

  const urgentWords = r.urgentKeywords.split(",").map((w) =>
    w.trim().toLowerCase()).filter(Boolean);
  const mediumWords = r.mediumKeywords.split(",").map((w) =>
    w.trim().toLowerCase()).filter(Boolean);

  urgentWords.forEach((k) => {
    const hits = countOccurrences(text, k);
    if (hits > 0) score += 2 * hits;
  });
  mediumWords.forEach((k) => {
    const hits = countOccurrences(text, k);
    if (hits > 0) score += hits;
  });

  let priority = "low";
  if (score >= r.urgentThreshold) priority = "urgent";
  else if (score >= r.mediumThreshold) priority = "medium";

  return { priority, reason, score };
}
```

7.3 User Experience and System Evaluation

Interfaces are role-tailored: Residents access services centrally, Reception handles front-desk efficiently, Security verifies visitors simply, and Admins use dashboards for oversight. Consistent design reduces learning curves versus manual methods.

User-experience outcomes can be interpreted across roles as follows:

Table 30. Role-Based User Feedback

User Role	Key Feedback	Benefit Demonstrated
Resident	Easy complaint submission.	Centralized service access.
Admin	AI reduces prioritizing workload.	Dashboard + Filtering tools.
Reception	Simple visitor verification.	Streamlined entry/exit logs.
Security	Faster parcel notifications.	Real-time resident updates.

The principal strengths of the implemented system are as follows:

- Comprehensive multi-portal architecture aligned with real operational roles.
- End-to-end traceability from request initiation through operational closure.

- Integrated notifications and dashboards that support situational awareness.
- AI-assisted complaint prioritization with configurable, explainable rules.
- Reporting capability through exportable PDF outputs.
- Modular design that supports extensibility and future scaling.

7.4 Limitations & Future Improvements

Despite the positive results, the following limitations were identified:

- Some features remain at prototype/demo level.
- AI model is rule-based and limited in adaptability.
- Security mechanisms require further enhancement.
- Lack of large-scale performance testing.
- Partial reliance on client-side validations.

Based on the discussion, the following improvements are recommended:

- Enhance AI using ML models.
- Implement stronger security mechanisms.
- Introduce analytics dashboards.
- Improve scalability for multi-apartment environments.
- Optimize user workflows and automation.

7.5 Final Insights

The results demonstrate that the Havelock Smart Resident Portal successfully achieves its objective of digitizing apartment management processes. The system improves efficiency, enhances communication, and introduces intelligent automation through AI-based prioritization. While certain limitations exist, the platform offers a solid basis for future development and real-world deployment, making it a valuable contribution to smart community management systems.

Chapter 08: Conclusion

This research addressed the need for integrated, digitized apartment-community operations in which residents, reception, security and admins share one coherent system instead of fragmented manual or informal channels. The objective was to design and implement Havelock Smart Resident Portal, a multi-portal web platform that supports end-to-end workflows for visitor management, parcel handling, complaints, billing visibility, notifications, and reporting, with role-based access aligned to real building operations.

The work resulted in a functional implementation comprising a shared backend with persistent data, dedicated portals for each stakeholder group, and a unified entry point for authentication. AI-assisted complaint prioritization was implemented through configurable, rule-based scoring, improving triage consistency and operational ordering of maintenance and service requests. Resident authentication via Google was integrated using JWT-based session handoff to the resident application, supporting modern sign-in expectations while remaining compatible, with password-based access elsewhere in the system. Email and in-app notifications were used to strengthen timelines and traceability across handoffs.

The evaluation and discussion showed that the platform reduces non-transparency and delay typical of traditional desk, gate and phone-based processes, and that it provides a practical basis for management oversight through structured data and exportable reports. At the same time, the study recognized limitations such as the gap between a demonstration-grade deployment and fully hardened production security, the value of deeper empirical measurement, and opportunities for further automation and analytics.

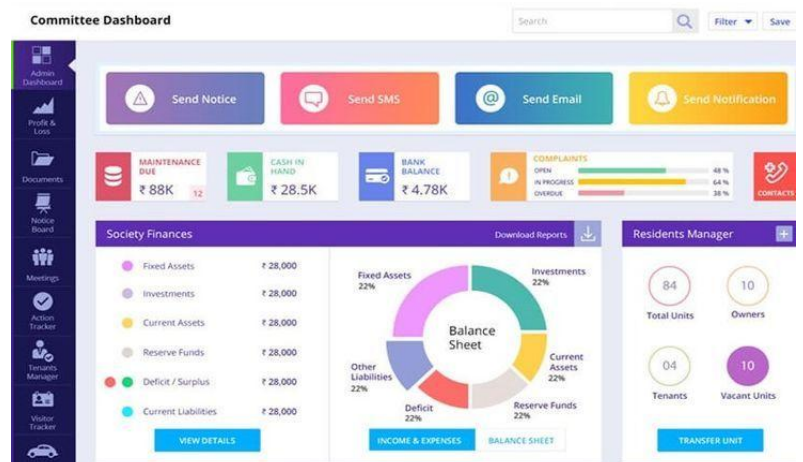
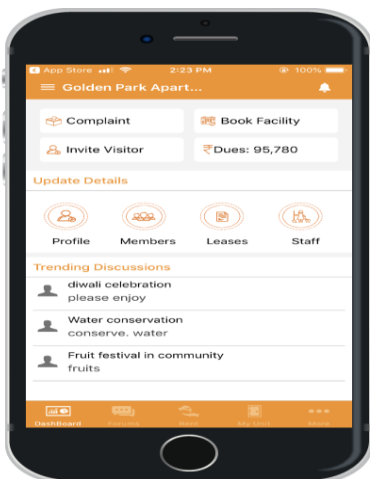
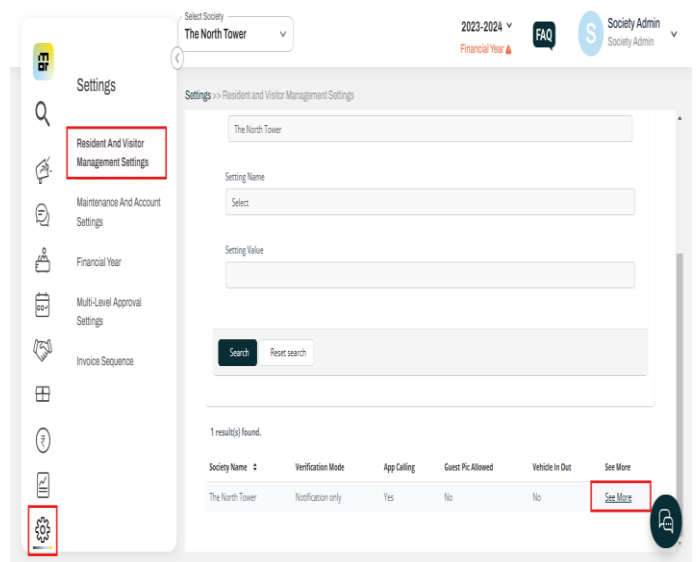
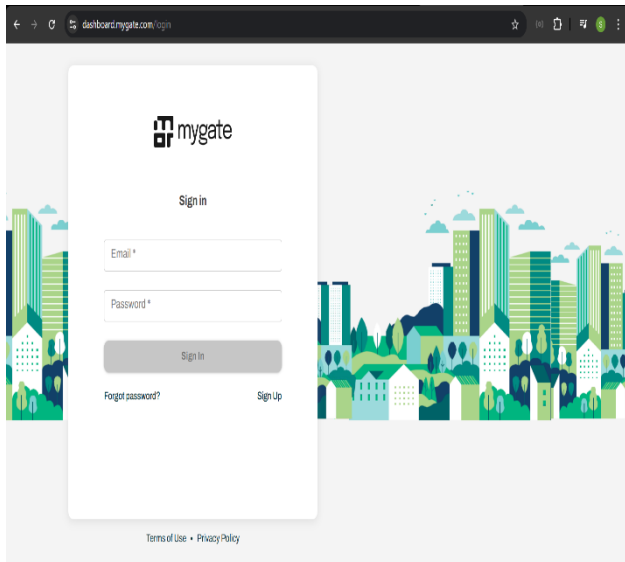
In summary, Havelock Smart Resident Portal demonstrates that a modular, multi-portal architecture combined with rule-based AI prioritization and integrated notifications can meaningfully support smart apartment operations. The outcomes validates the feasibility of the approach and establish a clear direction for future work in security hardening, scalability, richer analytics, and more advanced AI while preserving explainability and governance suitable for real-world building management.

References

- Puro, H., 2025. A Comprehensive Approach to Full-stack Application Design and Implementation. [online] Available at: <https://tinyurl.com/y76ac84f> [Accessed 18 April 2026].
- Sapkota, K., 2019. Artificial Intelligence in Property Management Automation – Technologies, Current Applications and Challenges. [online] Available at: <https://tinyurl.com/48b7yvh7> [Accessed 18 April 2026].
- Caliva, E., 2021. Development of a Web Application for Interactive Analysis of the Italian Oil Energy Sector Data. [online] Available at: <https://tinyurl.com/3y3um7zn> [Accessed 18 April 2026].
- React, 2024. Quick Start – React. [online] Available at: <https://react.dev/learn> [Accessed 19 April 2026].
- Node.js, 2025. Available at: <https://nodejs.org/en> [Accessed 19 April 2026].
- Cursor Blog RSS, 2025. Available at: <https://cursor.com/agents> [Accessed 19 April 2026].
- JavaScript, 1998. Tutorials and Guides – JavaScript. [online] Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide> [Accessed 19 April 2026].
- Joseph, K., 2021. Administration System for end-to-end Luxury Apartment Management Software. [online] Available at: <https://www.scienceopen.com/hosted-document?doi=10.14293/S2199-1006.1.SOR-.PPR0HQ1.v1> [Accessed 24 April 2026].
- Atlassian, 2019. Agile Best Practices and Tutorials. [online] Available at: <https://www.atlassian.com/agile> [Accessed 24 April 2026].
- Gray, D., 2022. React Login Authentication with JWT Access, Refresh Tokens, Cookies and Axios. Youtube. [online] Available at: <https://www.youtube.com/watch?v=nI8PYZNFtac> [Accessed 24 April 2026].
- Google Drive, 2026. Research Documents. [online] Available at: <https://drive.google.com/drive/folders/1ro7y55qzVqDN2GITJqYY1NR1rTUDw9UX?usp=sharing> [Accessed 24 April 2026].
- Setty, Y., 2022. Management System for an Apartment. [online] Available at: <https://opus.govst.edu/capstones/585/> [Accessed 24 April 2026].
- Kunawatsatit, P., 2007. Apartment Management System Case Study for Thaweelert Apartment. [online] Available at: <https://mfuir.mfu.ac.th/jspui/bitstream/123456789/456/1/49822-Fulltext.pdf> [Accessed 24 April 2026].

Appendixes

Appendix A – Similar Applications



Appendix B – Recording of the System Demonstration

<https://drive.google.com/file/d/1S74rWwFVvVkRUKRbip8GO20ytL8xlwdM/view?usp=sharing>

Appendix C – Project Poster

Havelock Smart Resident Portal

An AI-driven Apartment Management System

Introduction

Havelock **Smart Resident Portal** is a web-based application that customize for Havelock City apartment. It consists of four roles as Resident, Admin, Reception and Security. Havelock Smart Resident Portal will make easy daily activities inside the Havelock City.

Aim

To develop an AI-driven apartment management system with applied security gate control that increases communication, automation and safety within the Havelock City apartments.



Objectives

- To design and develop a smart web-based portal integrating resident, reception, security and admin functionalities.
- To develop an AI-based complaint prioritization system.
- To create digital visitor management with 24-hour pass.
- To apply online billing and tracking for transparency.

Features

-  AI Complaint Prioritization
-  24h Visitor Pass
-  Parcel Tracking
-  Online Billing
-  Inter-vehicle Booking

Methodology

- Agile Development Process with iterative sprints.
- System Development: **Node.js** backend with **React.js**, **Tailwind CSS**, TypeScript Frontend
- Project Planning: Gantt chart



mongoDB.

MongoDB is used as the database to store and manage all apartment management data securely, scalably and efficiently.

Implementation

- Frontend portals implemented using reusable component architecture.
- Backend exposes REST endpoints for all business modules.
- Mongoose models manage schema validation and database operations.
- Notifications and scoped views improve real-time user awareness.

Future Developments

- Integrate payment gateways and automated report scheduling.
- Expand mobile-first features and QR-based verification flows.
- Integrate E-mail and SMS notifications.
- Deploy the final project.



Student Name: MS Sheriff | Student ID: 2526110
Course: BSc (Hons) Software Engineering | Supervisor: Mrs. Rameesha Kankanamge